



Running and Securing Kubernetes Apps

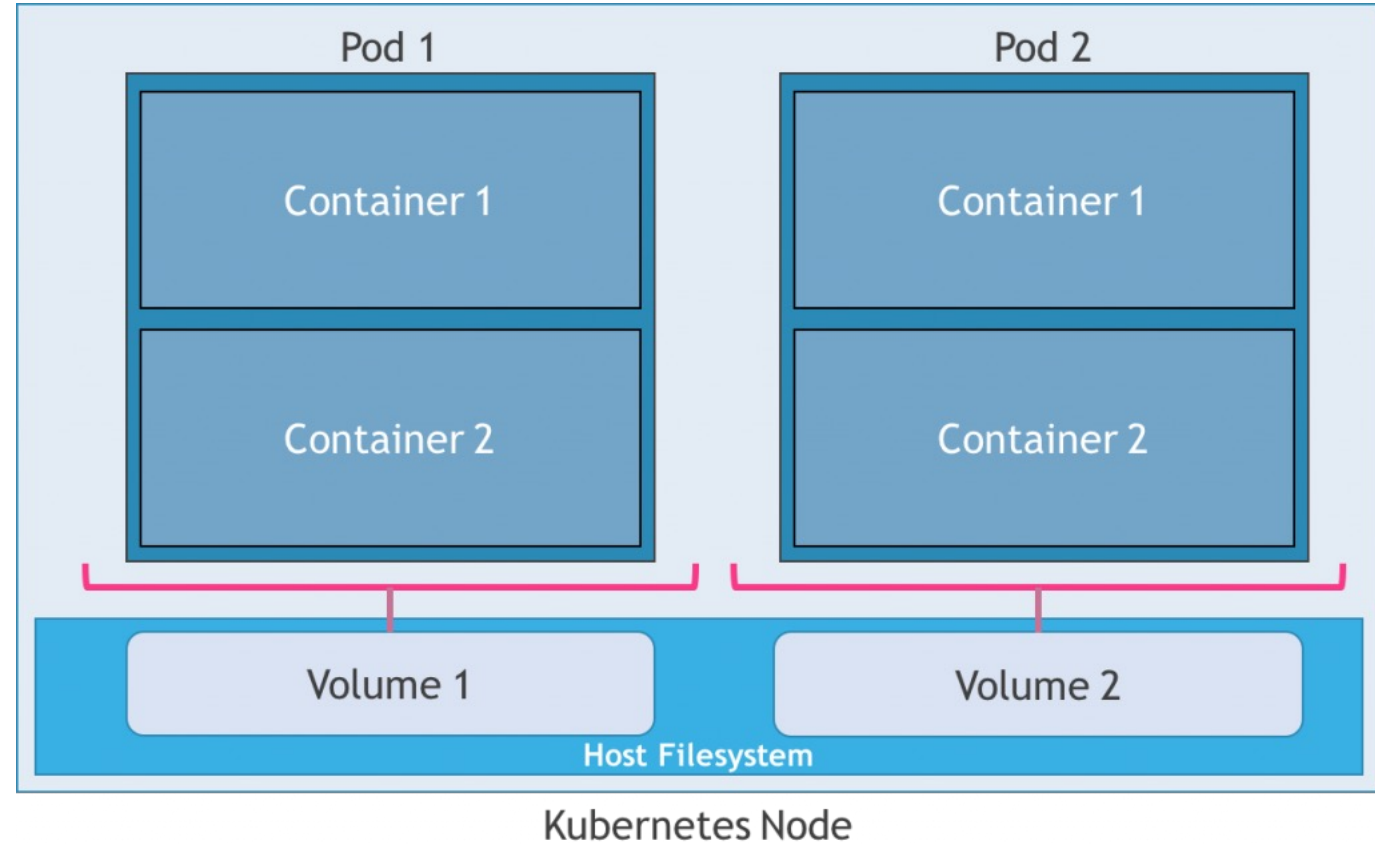


Persistent Volumes



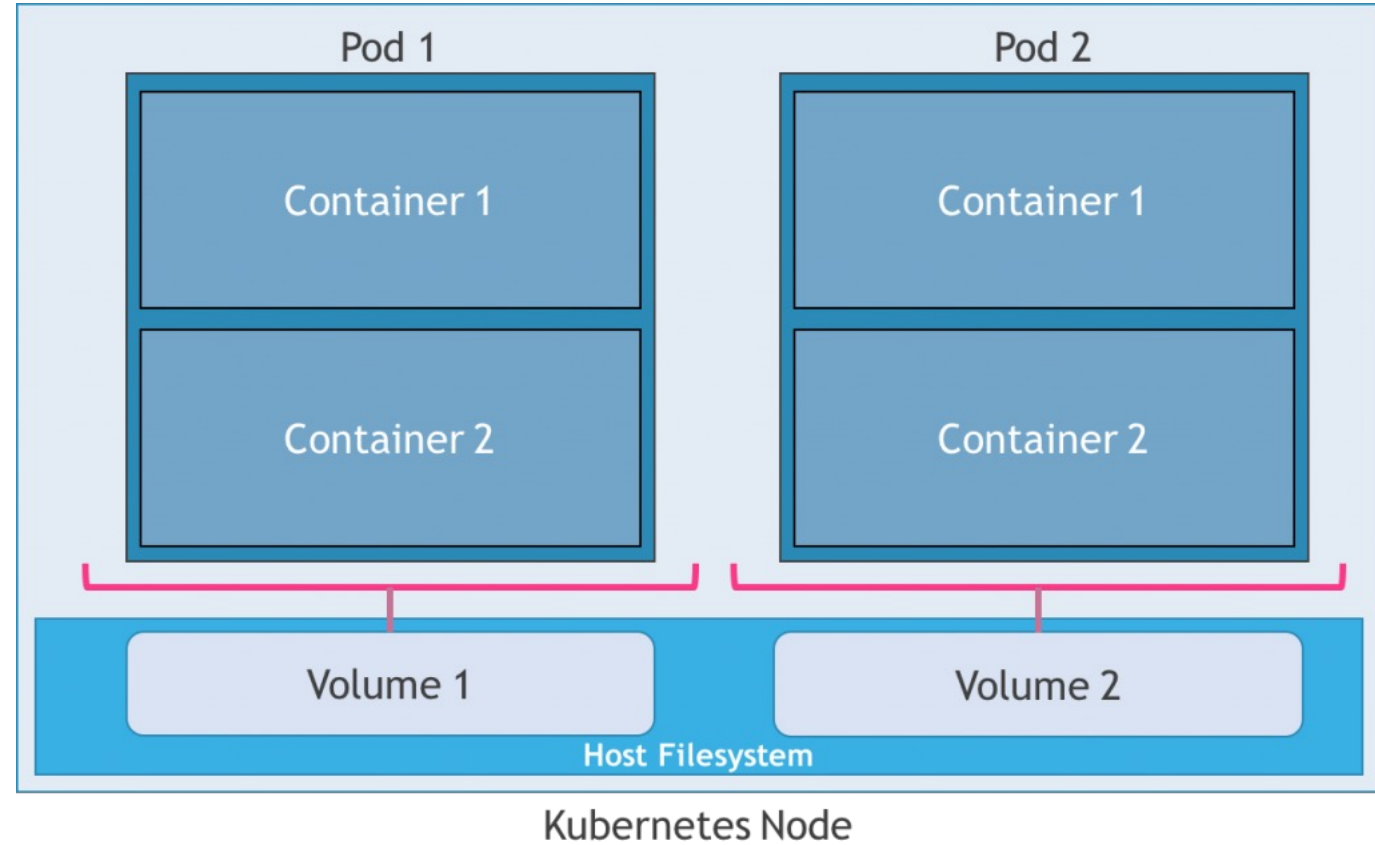
Host Based Storage

- Containers in a Pod share the same volume
- Host based storage
 - Local filesystem
- Removed when Pod is deleted
- Types of local storage
 - emptyDir
 - hostPath



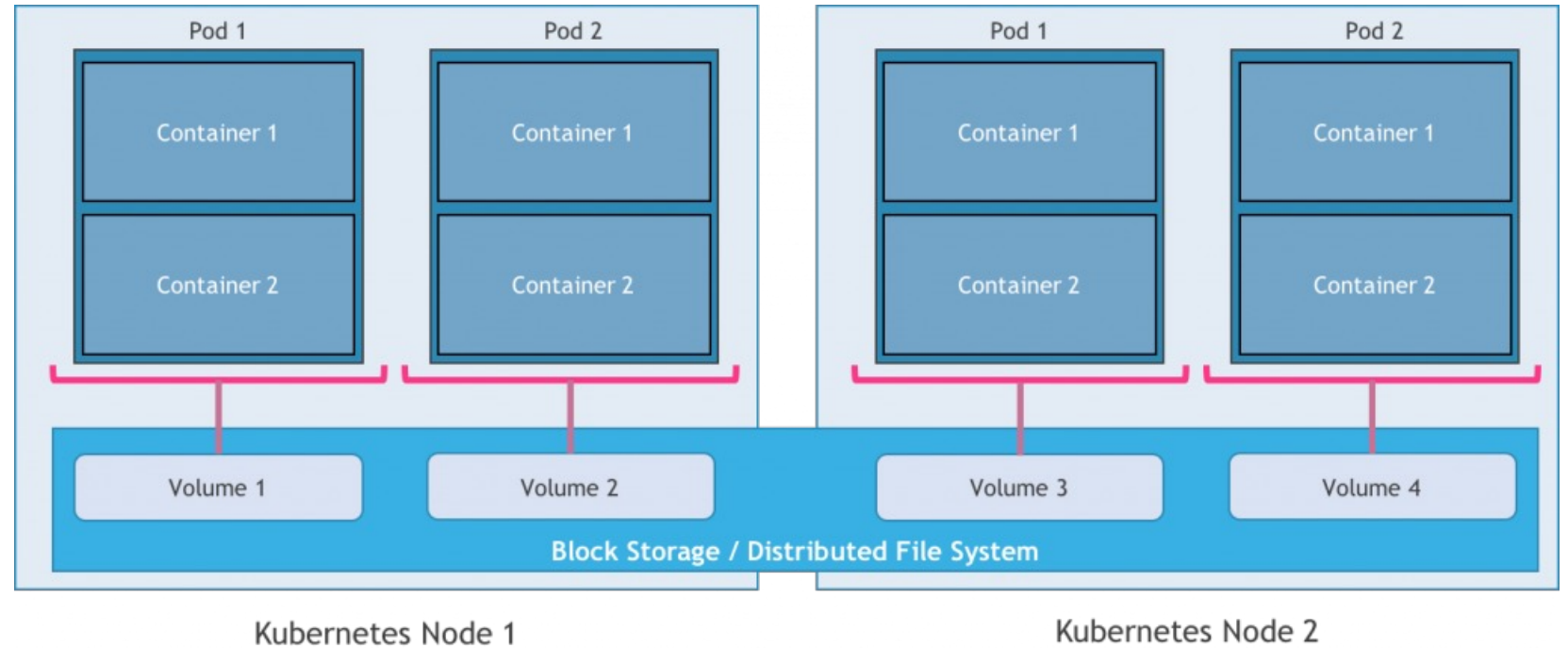
Host Based Storage

- Common use cases
 - Scratch disk
 - Store temp config data
- hostPath
 - Directories on host
 - Owned by root



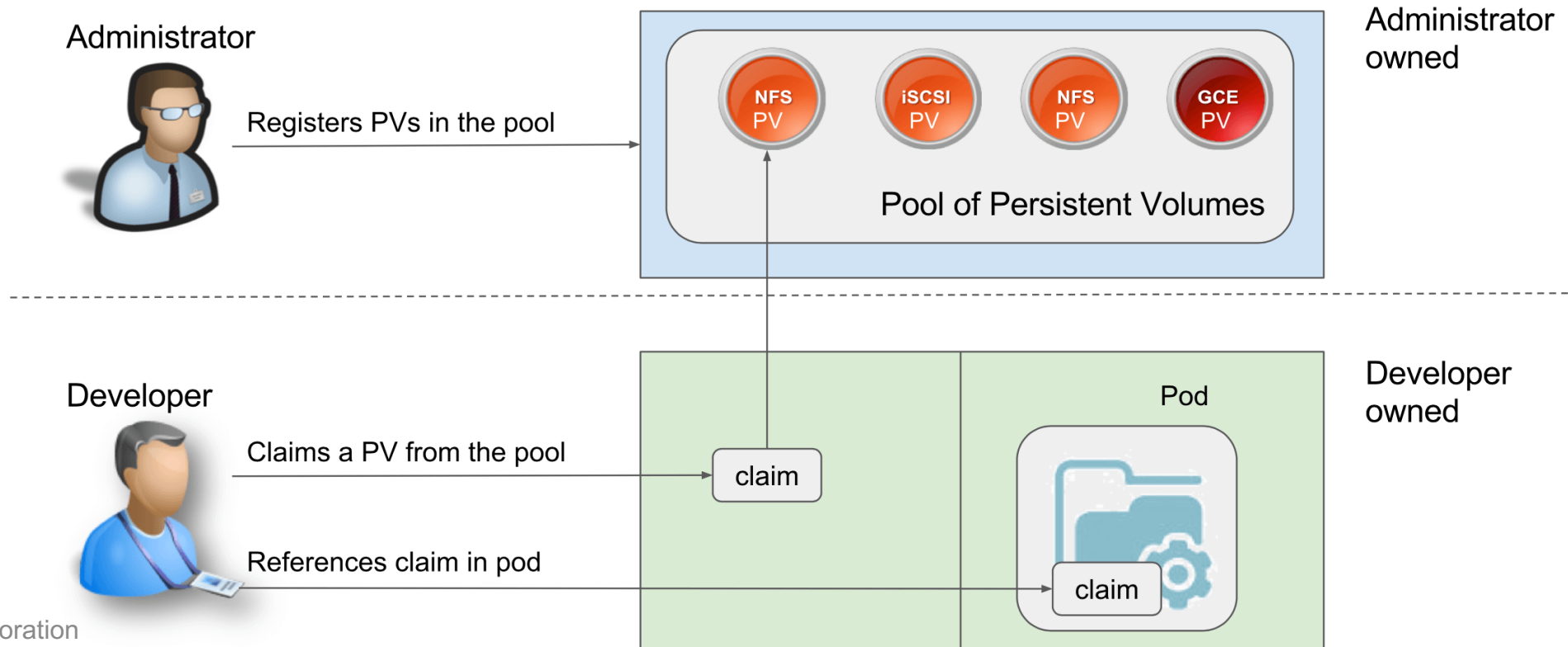
Non-Host Based Storage

- Common use cases
 - Persistent data
 - DB, Stateful apps
- EBS
- GCE Persistent Disks
- EFS, FSx
- NFS, NetApp, GlusterFS



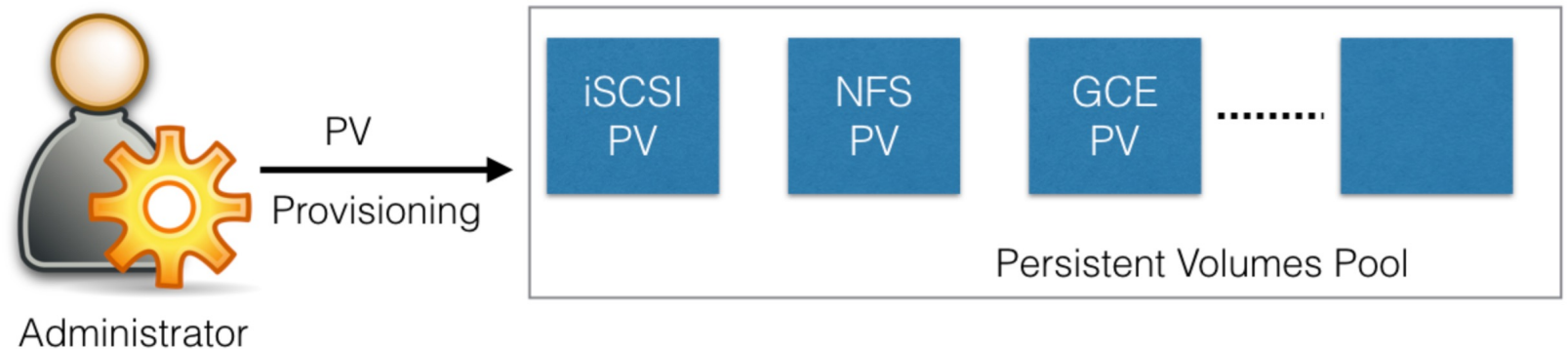
Persistent Volumes & Claims

- Operations creates PVs
 - Real storage details
 - Storage “LUN”
- Devs claim a PV
 - Self-service
 - Flexibility
 - Speed



Persistent Volumes

- Admin managed
- Quotas
- Access control
- Storage pools

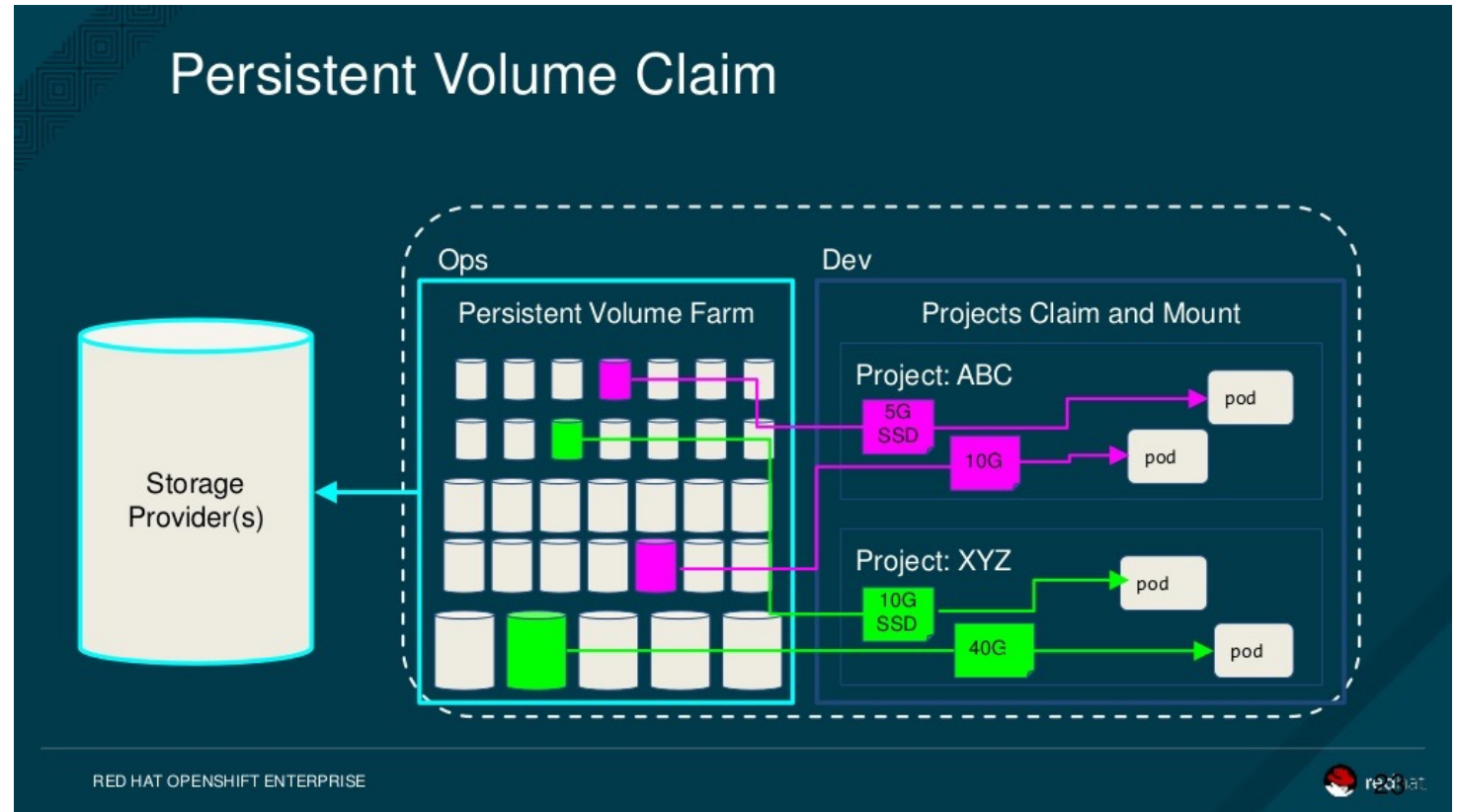


Persistent Volumes

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-aws
spec:
  capacity:
    storage: 12Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
  awsElasticBlockStore:
    fsType: "ext4"
    volumeID: "vol-f37a03aa"
```

Persistent Volume Claims

- Allows Devs to claim storage without worrying about management.
- Devs only see storage assigned to projects they are in.



Persistent Volume Claims

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongodb-pv-claim
labels:
  app: mongodb
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: rsvp-db
spec:
  replicas: 1
  template:
    metadata:
      labels:
        appdb: rsvpdb
    spec:
      containers:
        - name: rsvpd-db
          image: mongo:3.3
          ports:
            - containerPort: 27017
          volumeMounts:
            - name : mongodb-persistent-storage
              mountPath : /data/db
      volumes:
        - name: mongodb-persistent-storage
          persistentVolumeClaim:
            claimName: mongodb-pv-claim
```


Labs: Persistent Volumes



What are Monoliths?

Monolithic

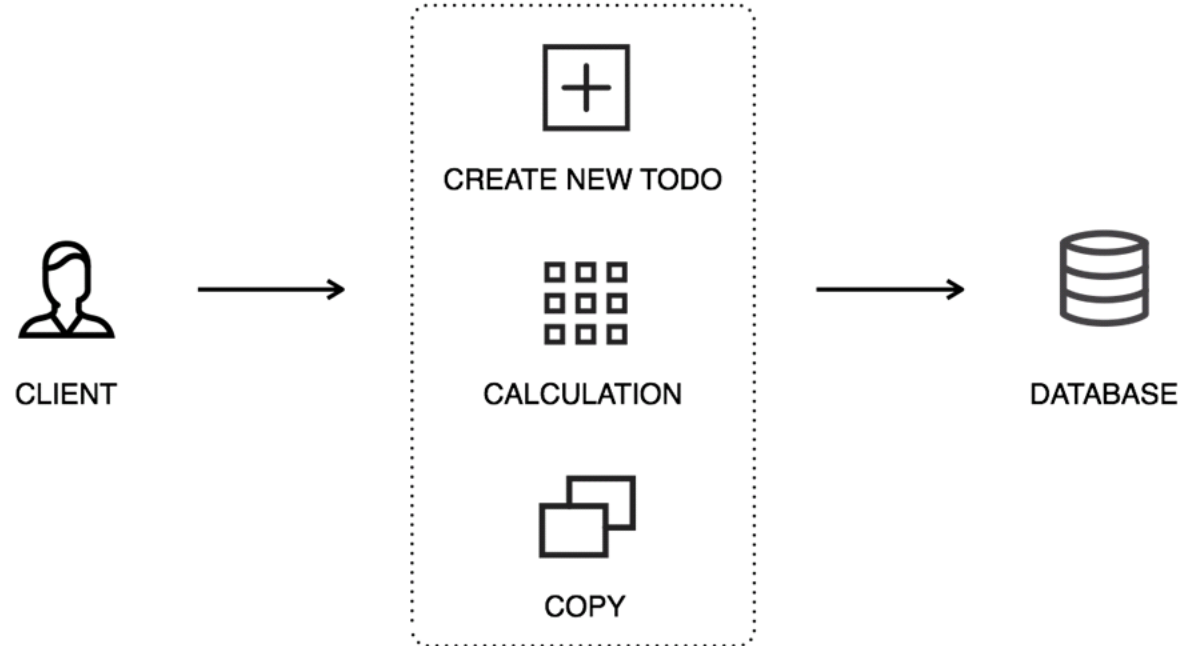
- Designed to be self contained
- Inter-connected services
- Tightly coupled components
- Must test all components if anything is changed
- Slower deploys
- Slower scaling

Monolithic

- Single development stack
- Changes are difficult
- Afraid to update

Monolithic

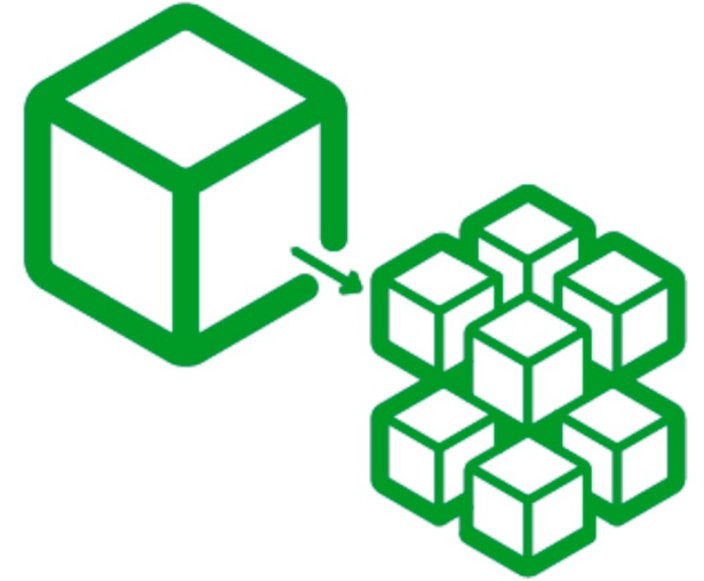
The Monolithic Architecture



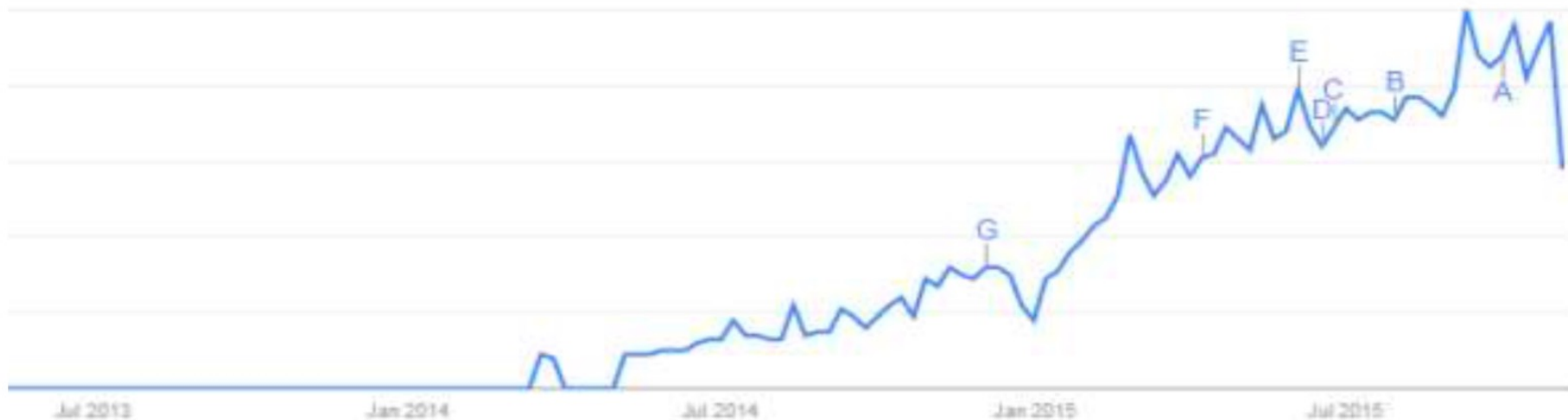
All services combined into one build,
written in the same language and application framework

What are Microservices?

What are Microservices?



The term microservices was coined in 2012 by a group of software architects



What are Microservices?

- Services communicate with each other over the network.
- Update services independently.
 - No need to change other services.
- Self-contained.
 - You can update the code without knowing anything about internals of other microservices.

What are Microservices?



adrian cockcroft
@adrianco

Follow

Replying to @kellabyte

@kellabyte @mamund I used to call what we did "fine grain SOA". So microservices is SOA with emphasis on small ephemeral components

5:16 PM - 10 Dec 2014

3 Retweets 5 Likes



7

3

5

**Functional decomposition of systems
into manageable and independently
deployable components,**

Microservice Architectures by Dr. Andreas Schroeder

**“Service-oriented architecture
composed of loosely coupled elements
that have bounded contexts”** - Adrian Cockcroft
(VP Cloud Architecture at AWS)

What are Microservices?

“In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API.”

- Martin Fowler

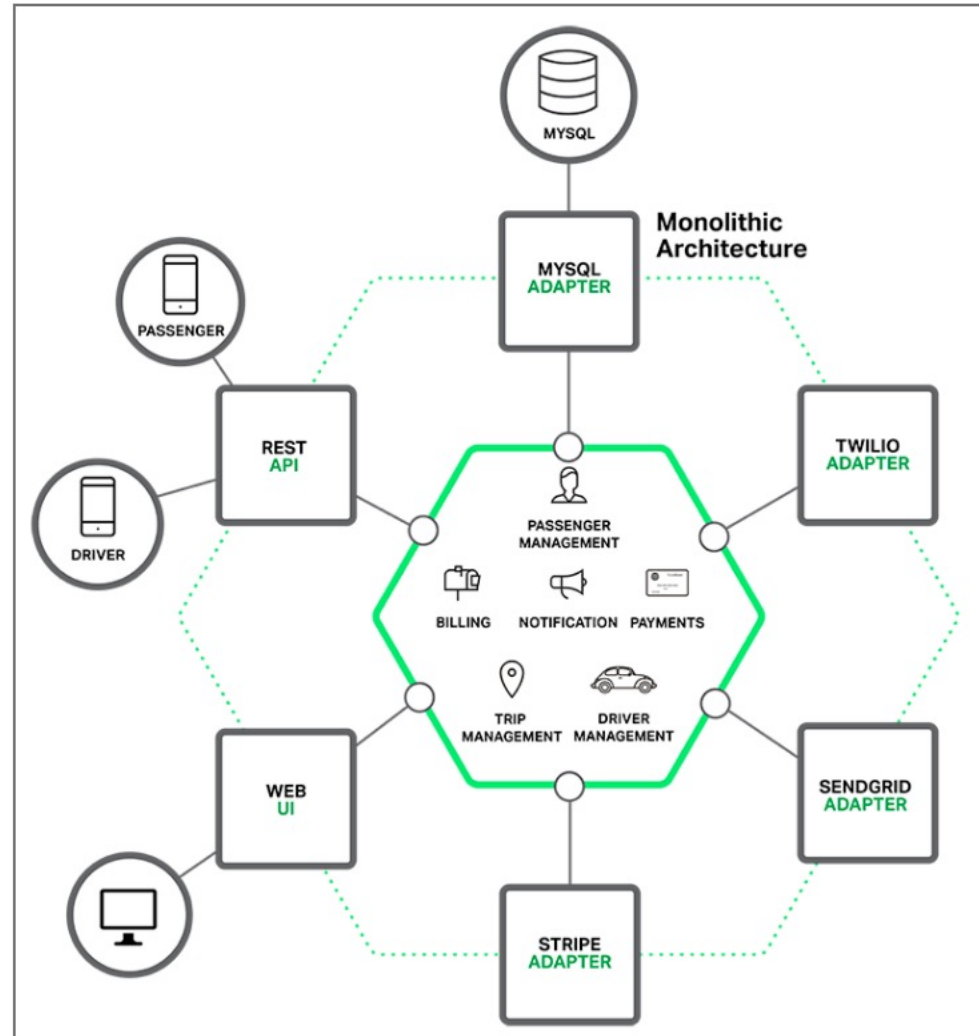
Guidelines for Microservices

- Make each one focused
 - “2 pizza teams”
- Treat each service as one app
 - Have one SCM repo, own pipeline, etc...
- Each service should be independent of other services
- Service owners are responsible for entire lifecycle
- Use lightweight protocols
- Use the right tool for the job

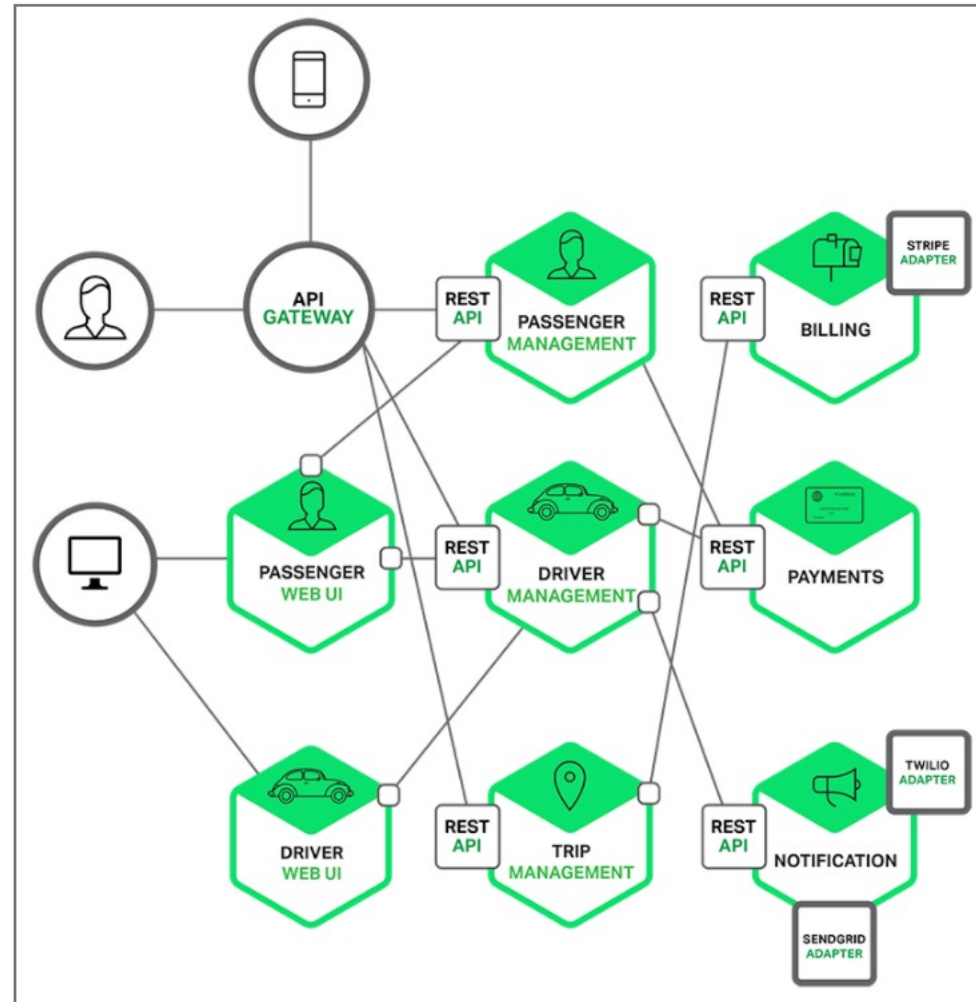
Guidelines for Microservices

- Each service should have its own datastore
- Failure will happen, design for it
- Automate everything!

Monolithic Architecture



Microservice Architecture

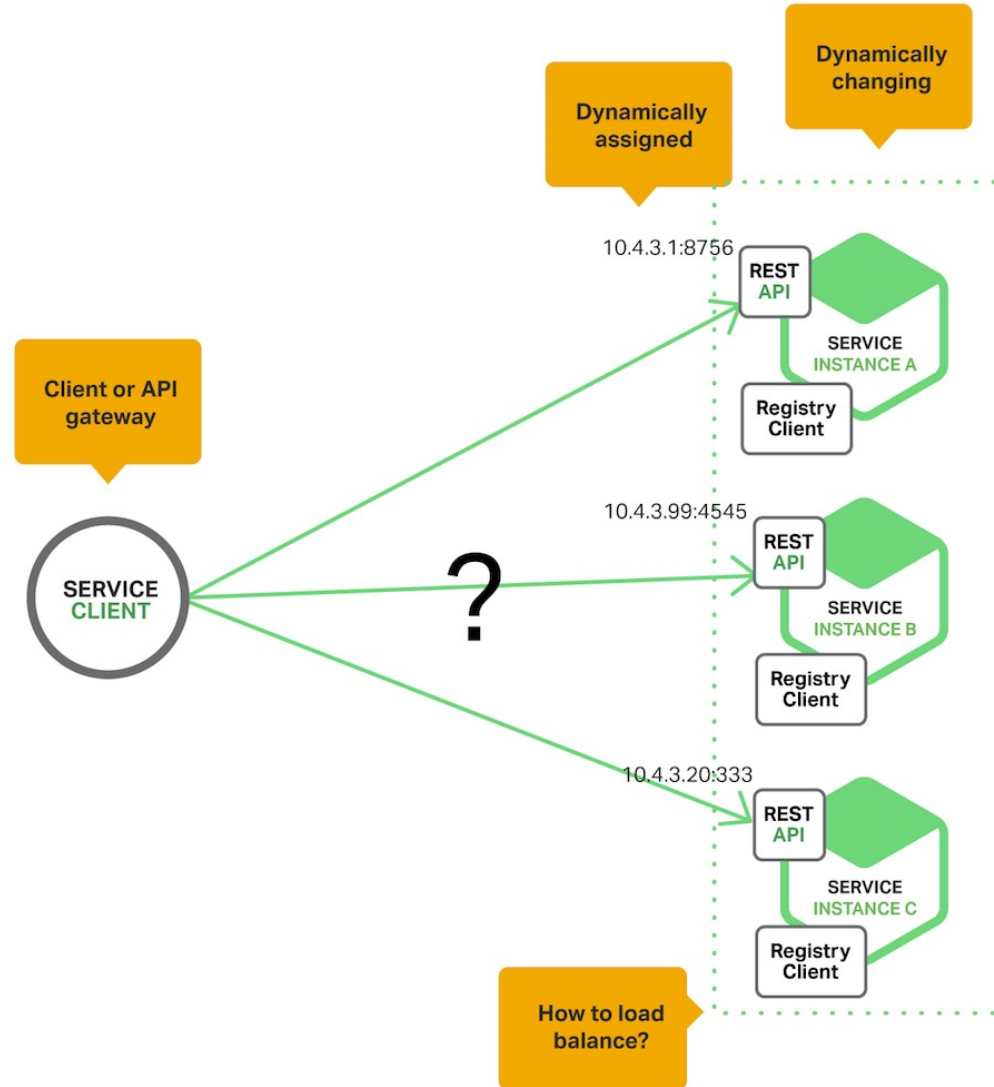


Microservices Challenges

- Service discovery & request routing
- Security and access control
- Efficiently scale up/down
 - connectivity
 - service resiliency
- Logging/Monitoring
- Canary/AB testing/rollouts/rollbacks

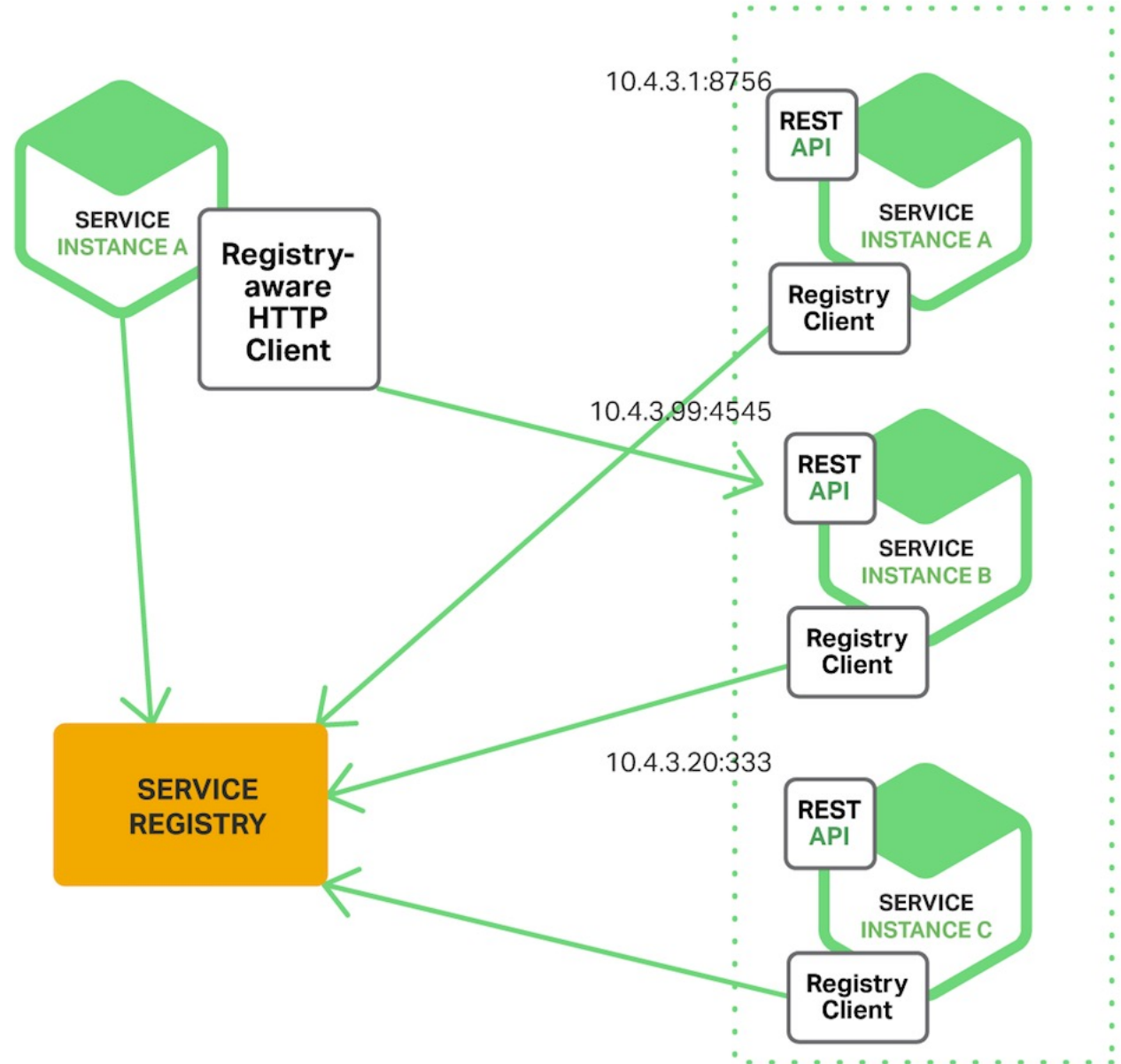
Service discovery

- Service discovery
 - Client side?
 - Server side?



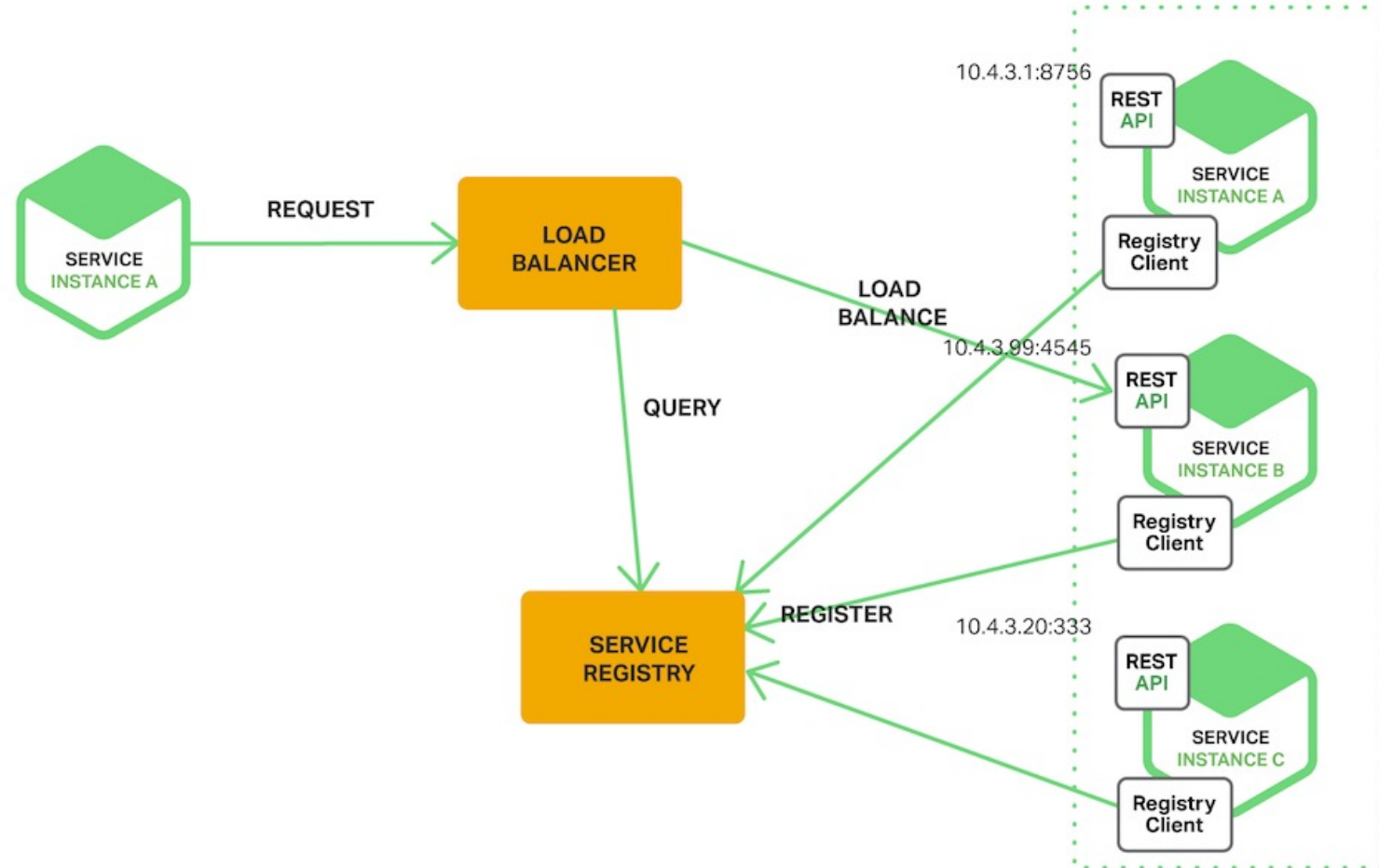
Service discovery

- Client side
 - Client responsible for finding services
 - Couples client and services requiring code changes



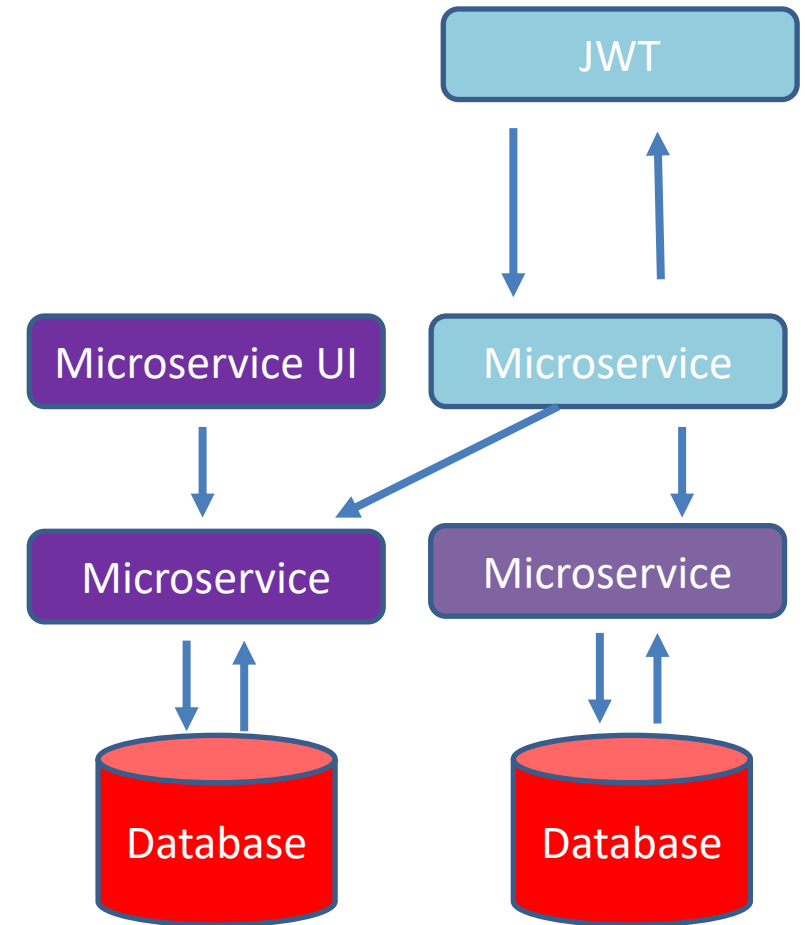
Service discovery

- Server side
 - ELB
 - Nginx & Consul
 - included in Kubernetes
 - Discovery abstracted away from client



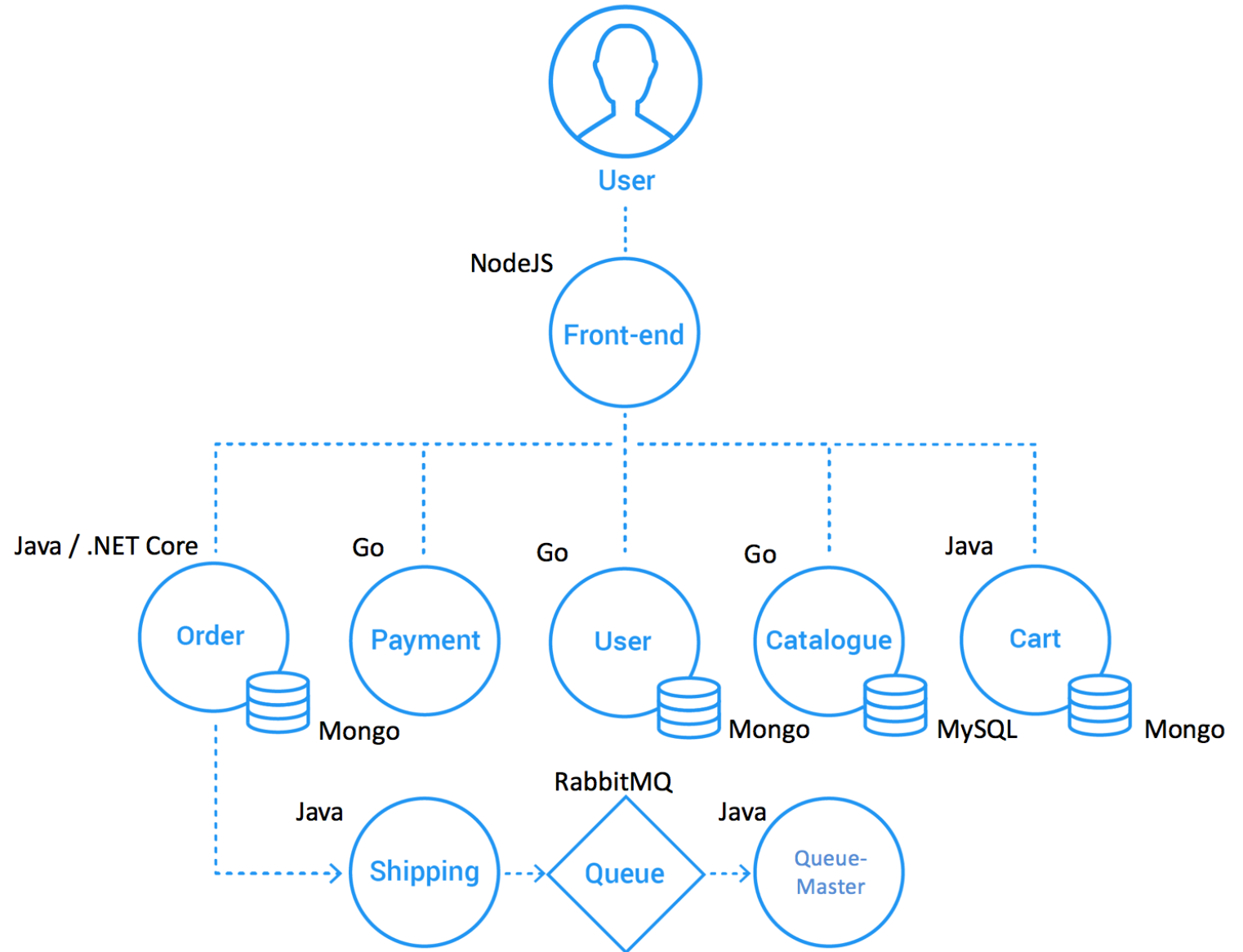
Security & Access Control

- Stateless
- JSON Web Token (JWT)
- API Gateway usually handles auth



Sock Shop

- Go
- Spring Boot
- Node.js
- RabbitMQ
- MySQL
- Mongo



Kubernetes POD Scheduling



Pod Scheduling Overview

kube-scheduler on the leader node assigns new pod requests to the appropriate worker nodes

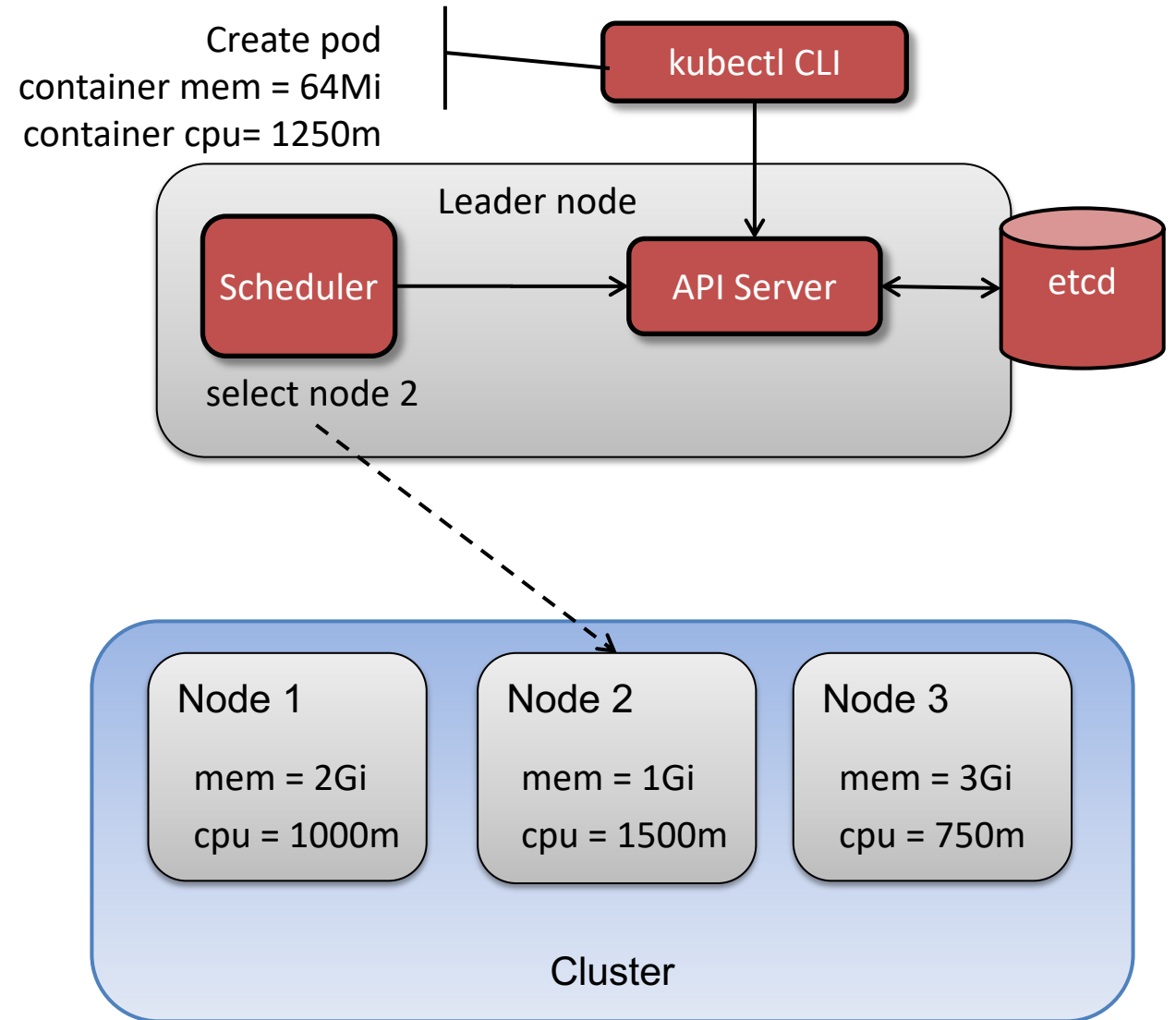
The default scheduler takes into account:

- Available node CPU/RAM
- Resource requests from new pod – sum of resource requests of pod containers

The default scheduler will automatically

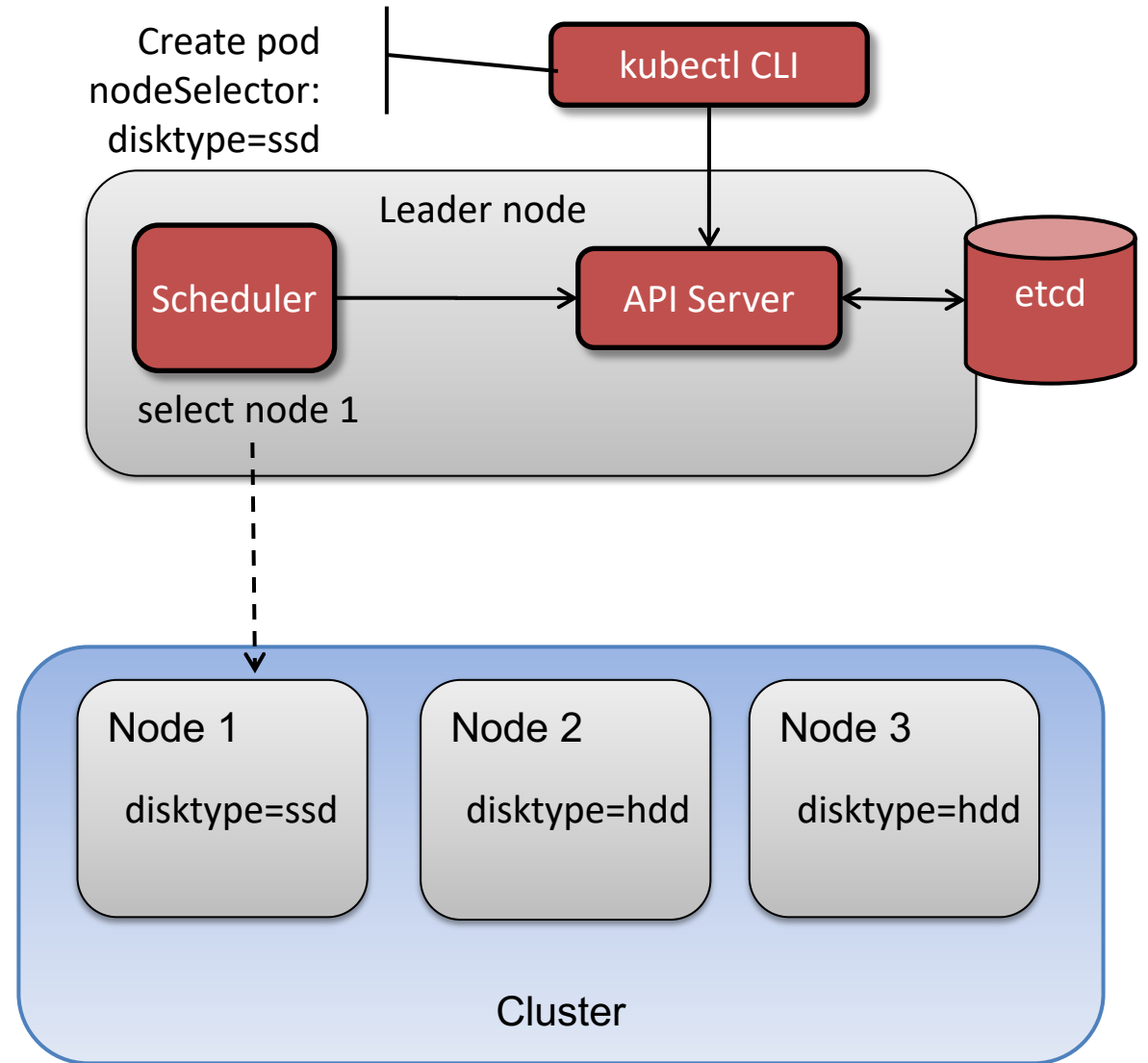
- Schedule a pod on a node with sufficient free resources
- Spread pods across nodes in the cluster

Can specify custom schedulers for pods



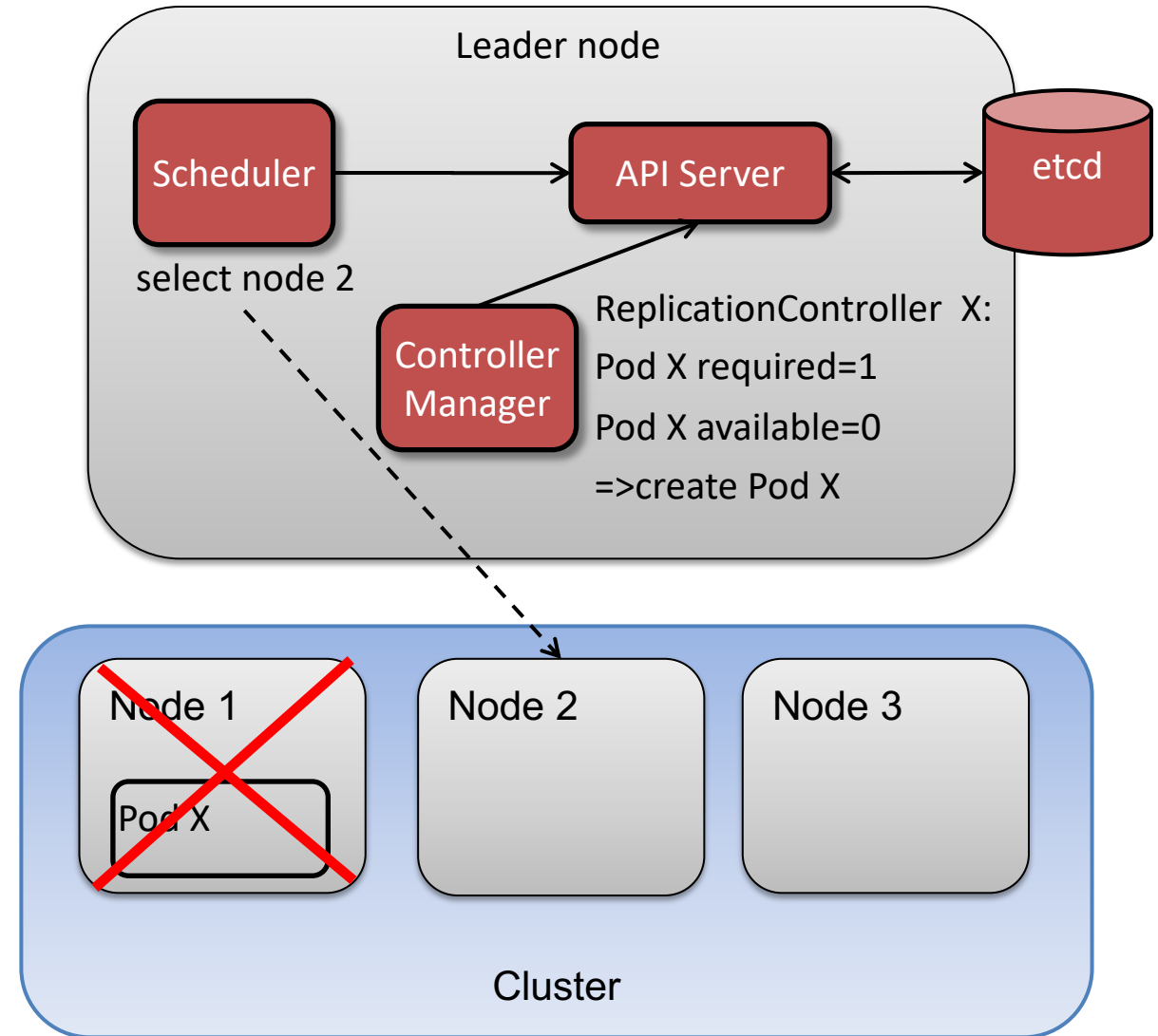
Controlling Pod Scheduling with Default Scheduler

- Nodes carry labels indicating topology and other resource notes
- Users can require pods to be scheduled on nodes with specific label(s) via a `nodeSelector` in container spec



Scheduling Pod Re-creation Driven by Control Loops

- kube-scheduler performs same node selection operation when new pod created due to e.g. node loss
- kube-controller-manager runs controllers like ReplicationController managing number of pod instances available
- kube-controller-manager will initiate request for new pods as needed, which will be scheduled by kube-scheduler per pod/container spec



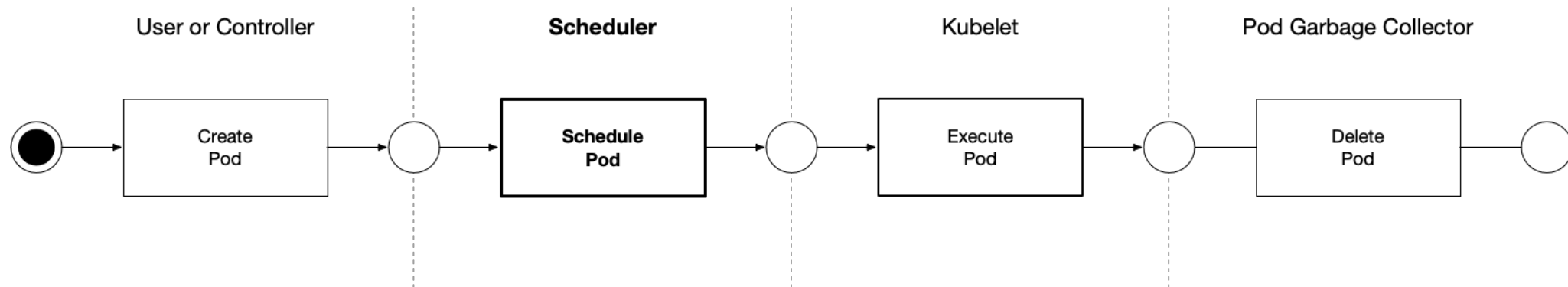
Kubernetes Scheduling

Node Selector

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    env: test
spec:
  container:
  - name: nginx
    image: nginx
    nodeSelector:
      disktype: ssd
```

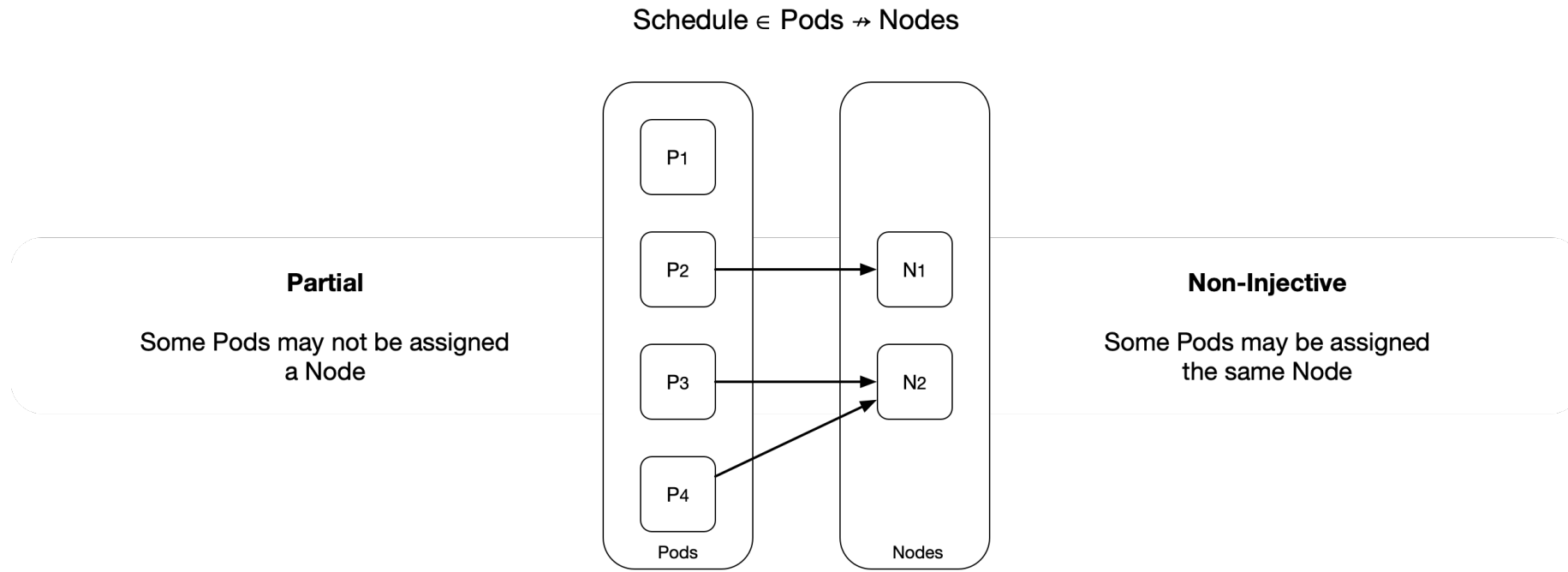
Scheduling flow

- **Scheduler (kube-scheduler):** selects nodes for newly created pods to run on, this is called a "placement"
 - Placement: a partial, non-injective assignment of a set of Pods to a set of Nodes.



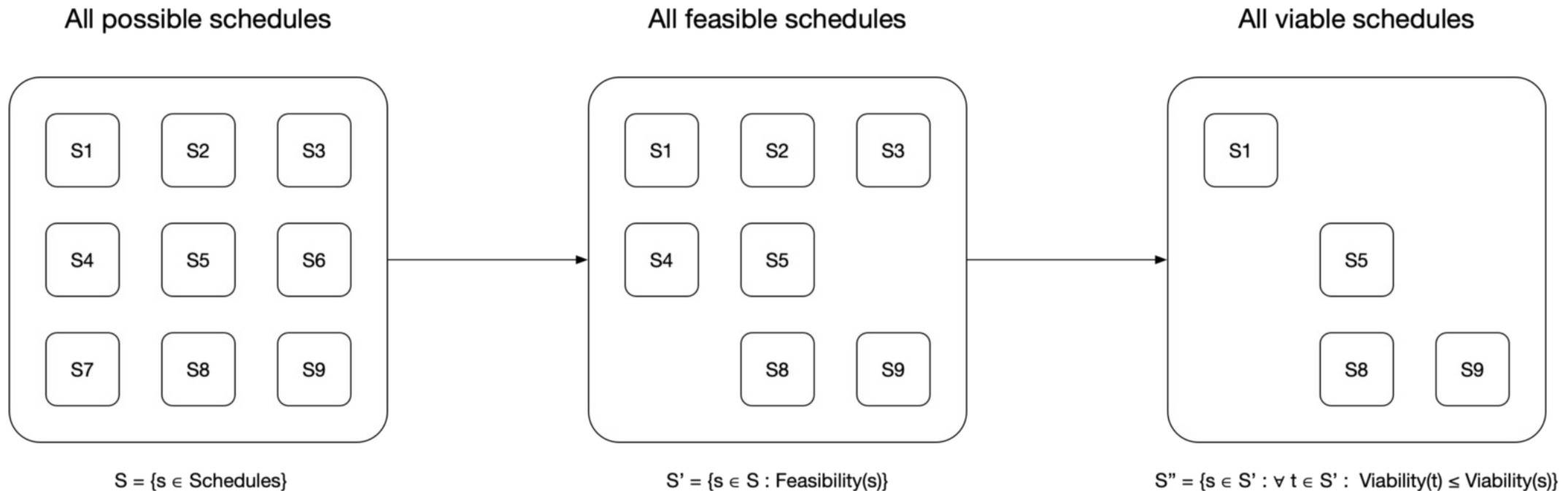
Scheduler details

- **Scheduler (kube-scheduler)**: selects nodes for newly created pods to run on, this is called a "placement"
 - Placement: a partial, non-injective assignment of a set of Pods to a set of Nodes.



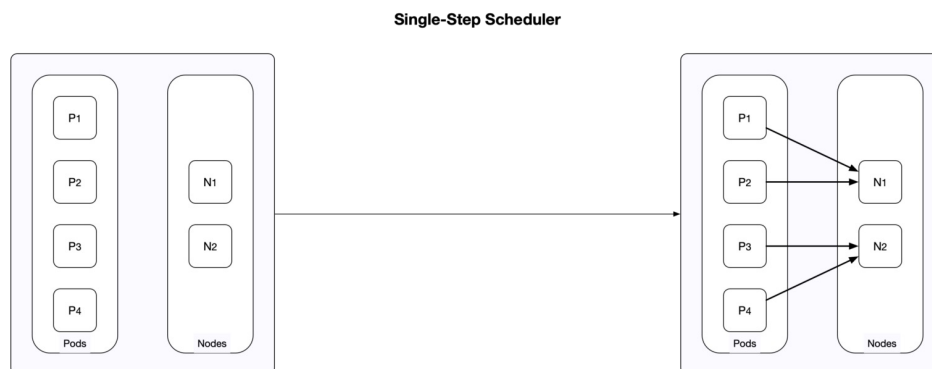
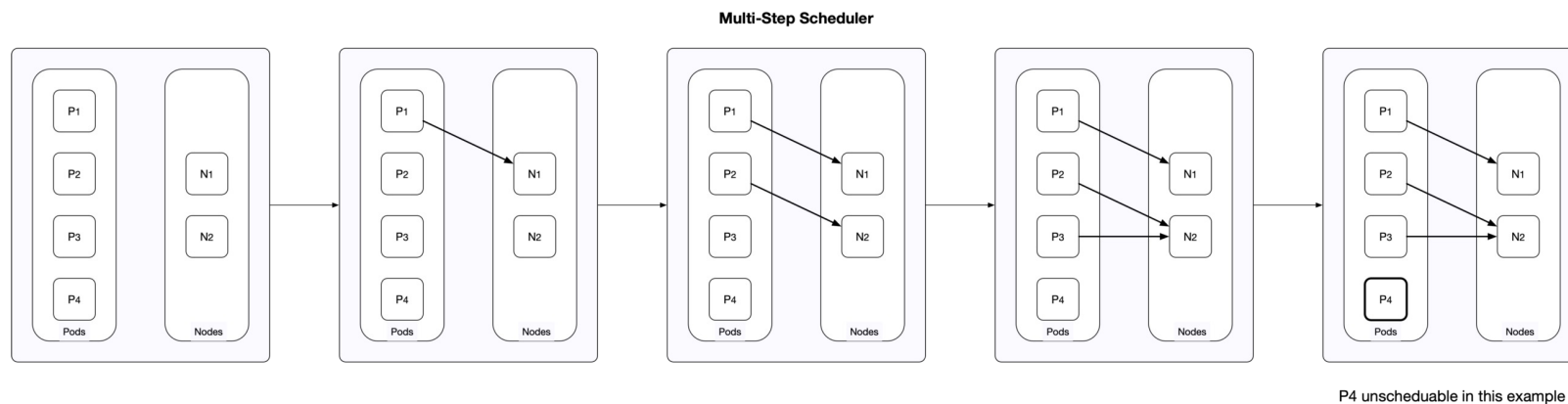
Scheduler decisions

- **Scheduler (kube-scheduler)**: selects nodes for newly created pods to run on, this is called a "placement"
 - Placement: a partial, non-injective assignment of a set of Pods to a set of Nodes.

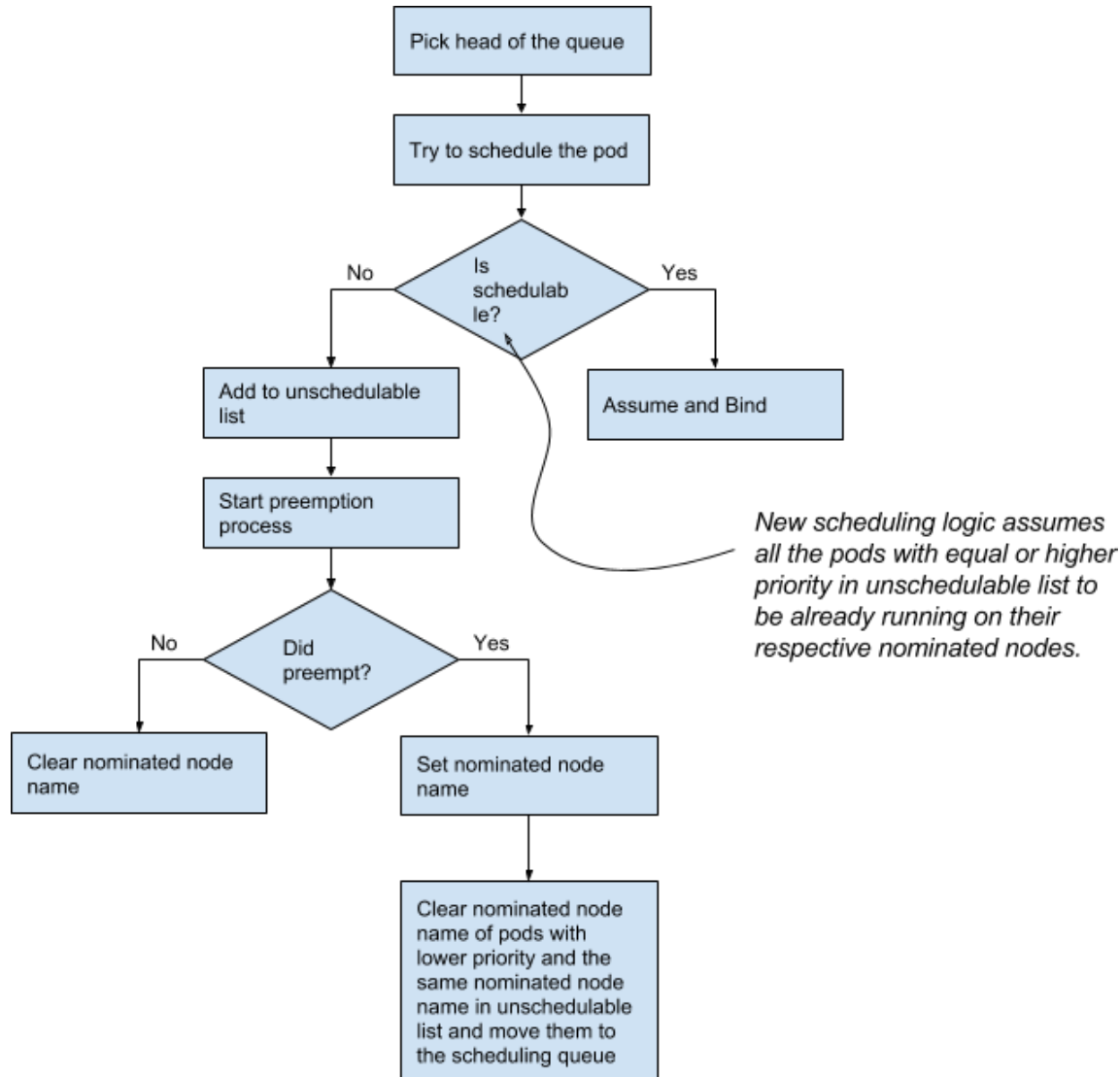


Scheduler decisions

- **Multi-step scheduler**
 - local optimum instead of global optimum



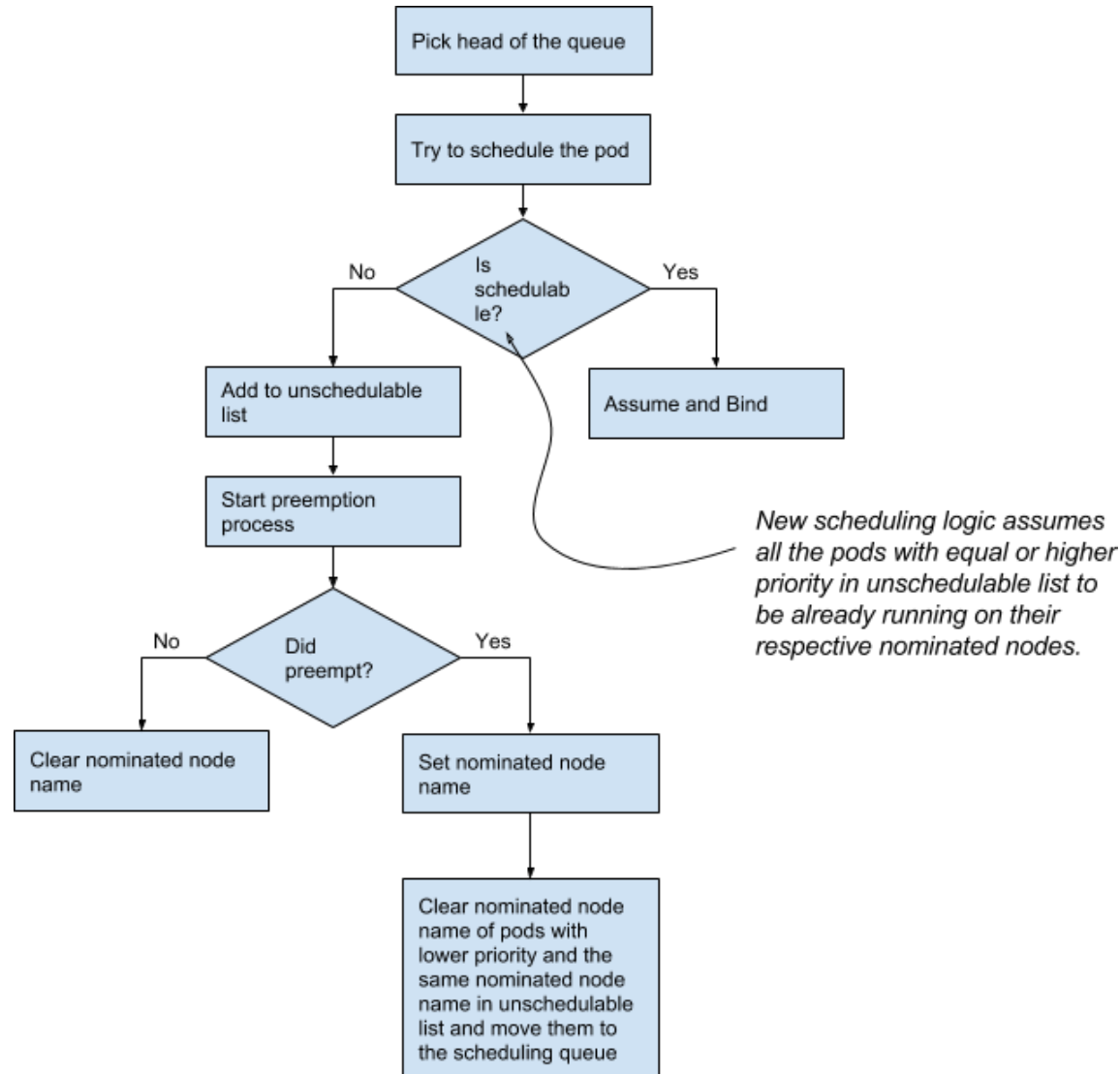
Scheduler preemption



- **Preemption**

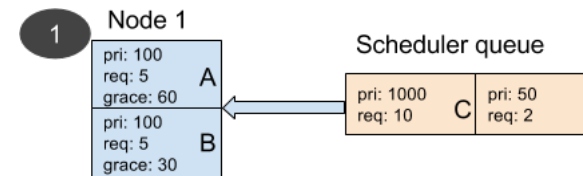
- Lower priority Pods are destroyed to free up resources for higher priority Pods.

Scheduler preemption



- **Example**

- 1 node in the cluster with capacity 10 units
- 2 pods (A,B) running on the node with priority 100 and each using 5 units
- Scheduler has 2 pods (C,D) in queue. Pod C needs 10 units and priority is 1000. Pod D needs 2 units and its priority is 50
- Scheduler determines that Pod C has highest priority and destroys (A,B) so it can schedule.



Kubernetes Advanced Pod Scheduling



Kubernetes Scheduling

- Node Selector
- Node Affinity
- Pod Affinity
- Pod Anti Affinity
- 3rd party schedulers

Kubernetes Scheduling

- Node affinity/anti-affinity
 - generalizing the nodeSelector feature
- Node taints
 - prevent scheduling of pods to nodes unless pod 'tolerates' the taint
- Pod affinity/anti-affinity
 - control relative placement of pods

Node Affinity

The affinity/anti-affinity feature greatly expands the types of constraints you can express.

The key enhancements are:

- More expressive language (not just "AND of exact match")
- Rule is "soft"/"preference" so if it doesn't match Pods are still scheduled.

Kubernetes Scheduling

Node Affinity

```
affinity:  
  nodeAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: In  
            values: ["us-central1-a"]
```


Kubernetes Scheduling

Node Affinity

```
affinity:  
  nodeAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: "failure-domain.beta.kubernetes.io/zone"  
            operator: In  
            values: ["us-central1-a"]
```

Kubernetes Scheduling

Weight example

```
affinity:
  nodeAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 5
        preference:
          - matchExpressions:
              - key: env
                operator: In
                values:
                  - dev
            preferredDuringSchedulingIgnoredDuringExecution:
              - weight: 1
                preference:
                  - matchExpressions:
                      - key: team
                        operator: In
                        values:
                          - engineering
```

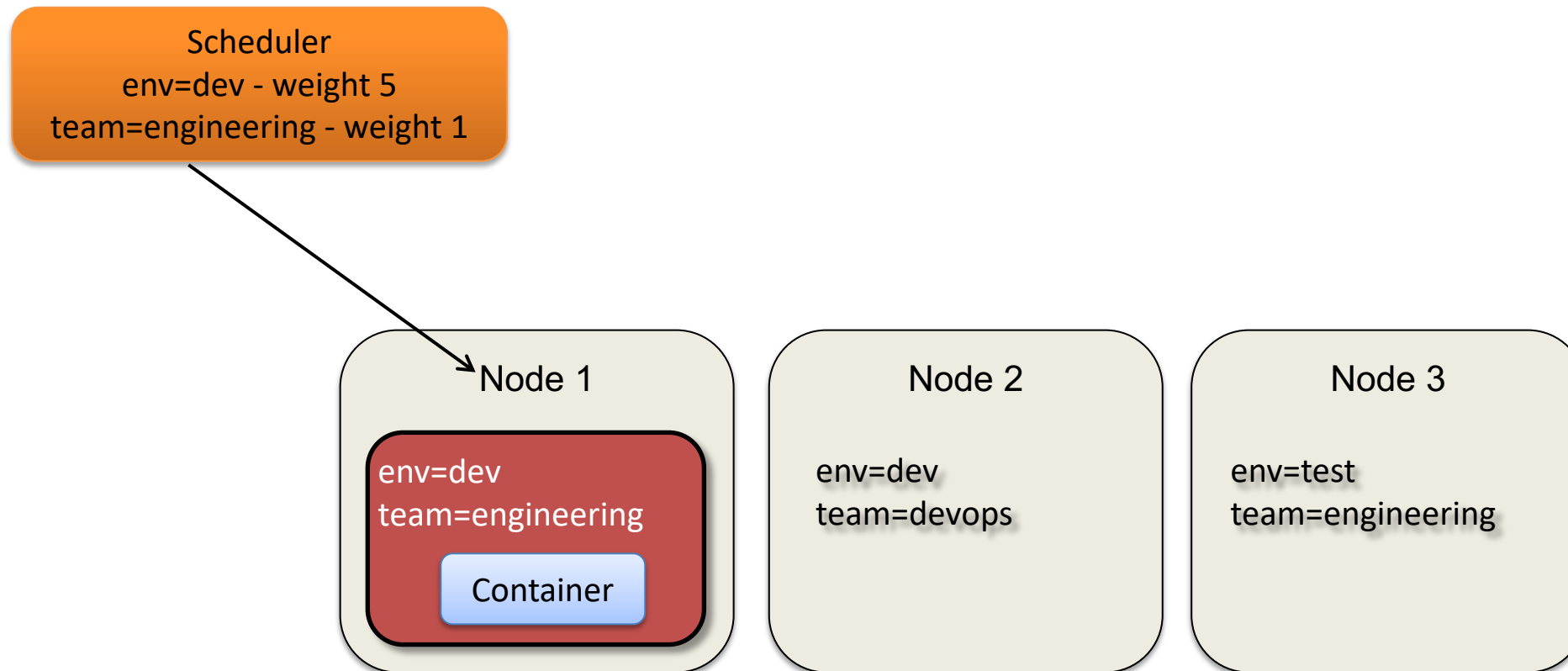
Node Affinity

Kubernetes supports "weighted scheduling"

- The higher this weighting the more weight is given to that rule.
- Scoring is done by summarizing weighting per node.
 - 2 rules weights 1 and 5
 - If both match = score of 6
 - If only 1 rule matches = score of 1
- Node with highest total weight score receives Pod.

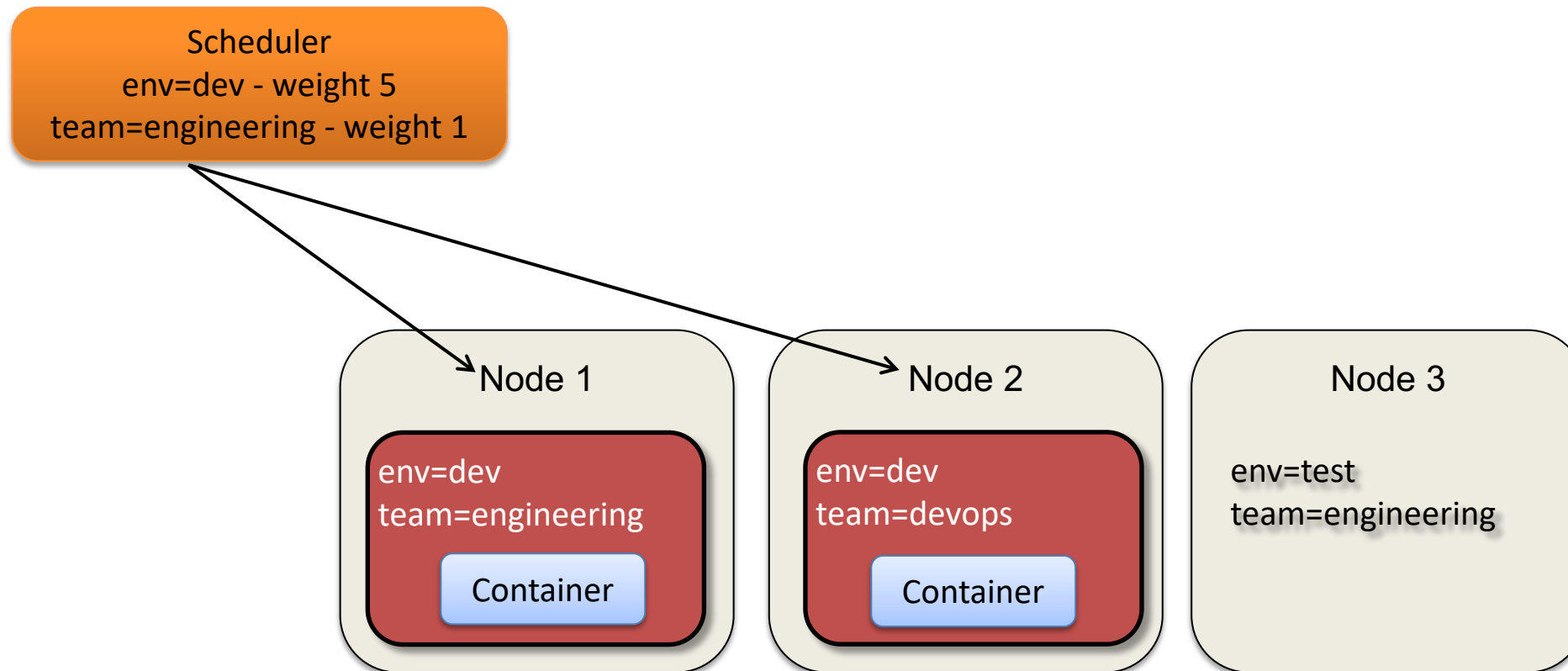
Node Affinity: Weight

- Schedule Pods onto nodes that have labels
 - env = dev
 - team = engineering



Node Affinity: Weight

- Schedule Pods onto nodes that have labels
 - env = dev
 - team = engineering



Operators

Valid operators

- In
- NotIn
- Exists
- DoesNotExist
- Gt (Greater Than)
- Lt (Less Than)

Node Taints

Allows you to mark ("taint") a node

- No pods can be scheduled onto tainted nodes
- Useful when all or most PODs should not be scheduled on a node
- Mark your master node as schedulable only by Kubernetes system components
- Keep regular pods away from nodes that have special hardware so as to leave room for pods that need the special hardware.

Node Tolerations

To allow a POD to be scheduled onto a 'tainted' node it must have:

```
tolerations:  
- key: "key"  
  operator: "Equal"  
  value: "value"  
  effect: "NoSchedule"
```

Pod Affinity

- Define how pods should be placed relative to one another
- Spread or pack pods within a service or relative to pods in other services

```
affinity:  
  podAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
        topologyKey: failure-domain.beta.kubernetes.io/zone
```

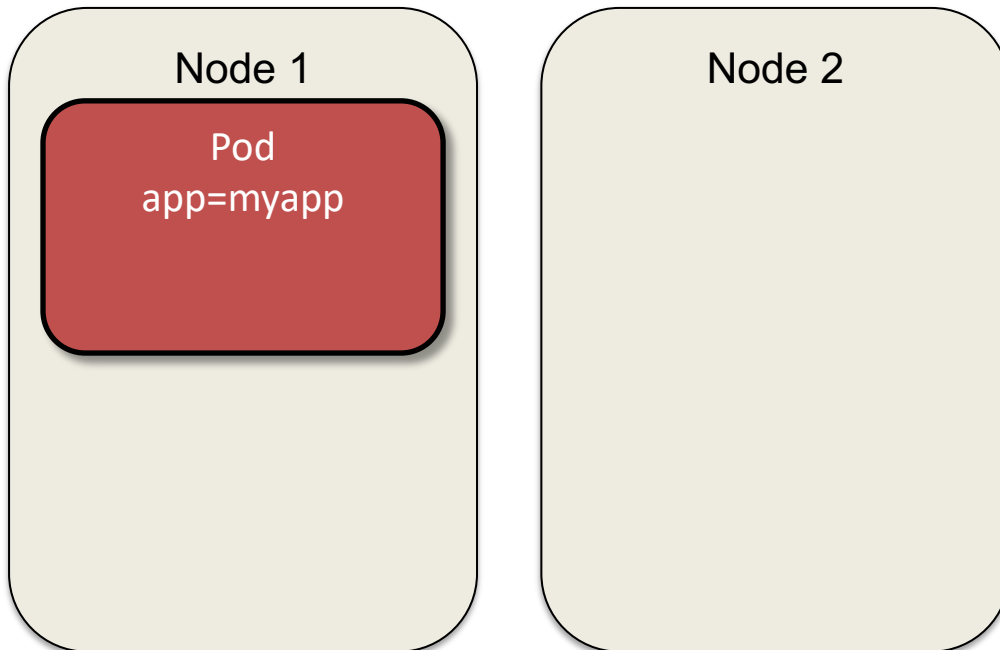
Pod Affinity

- Define how pods should be placed relative to one another
- Spread or pack pods within a service or relative to pods in other services

```
affinity:  
  podAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
        topologyKey: failure-domain.beta.kubernetes.io/zone
```

Pod Affinity

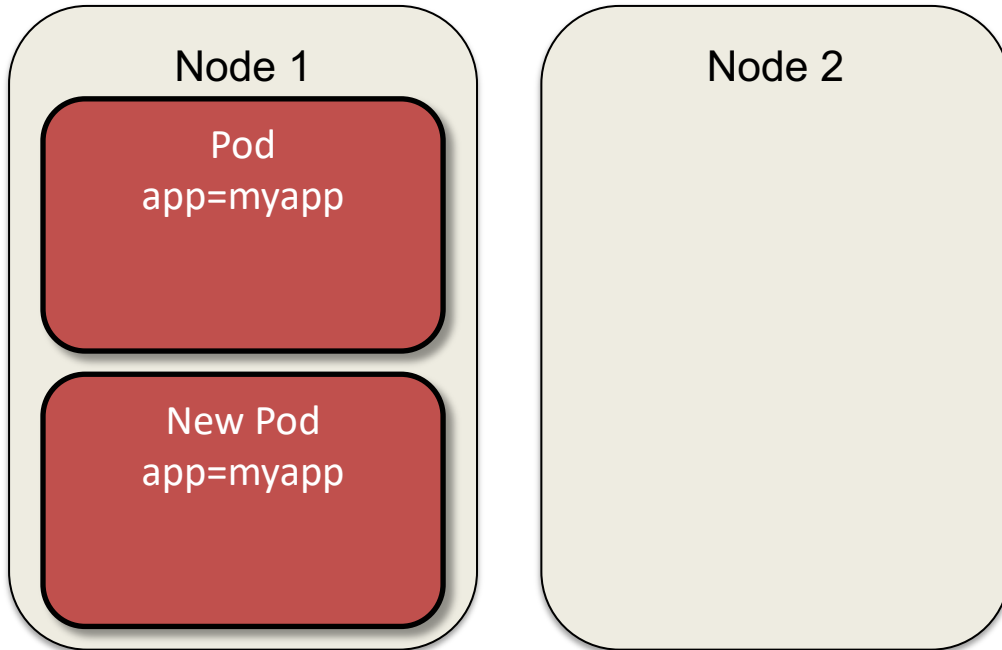
- Schedule Pods onto nodes that have same labels
 - app=myapp
 - operator: In
 - kubernetes.io/hostname



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

Pod Affinity

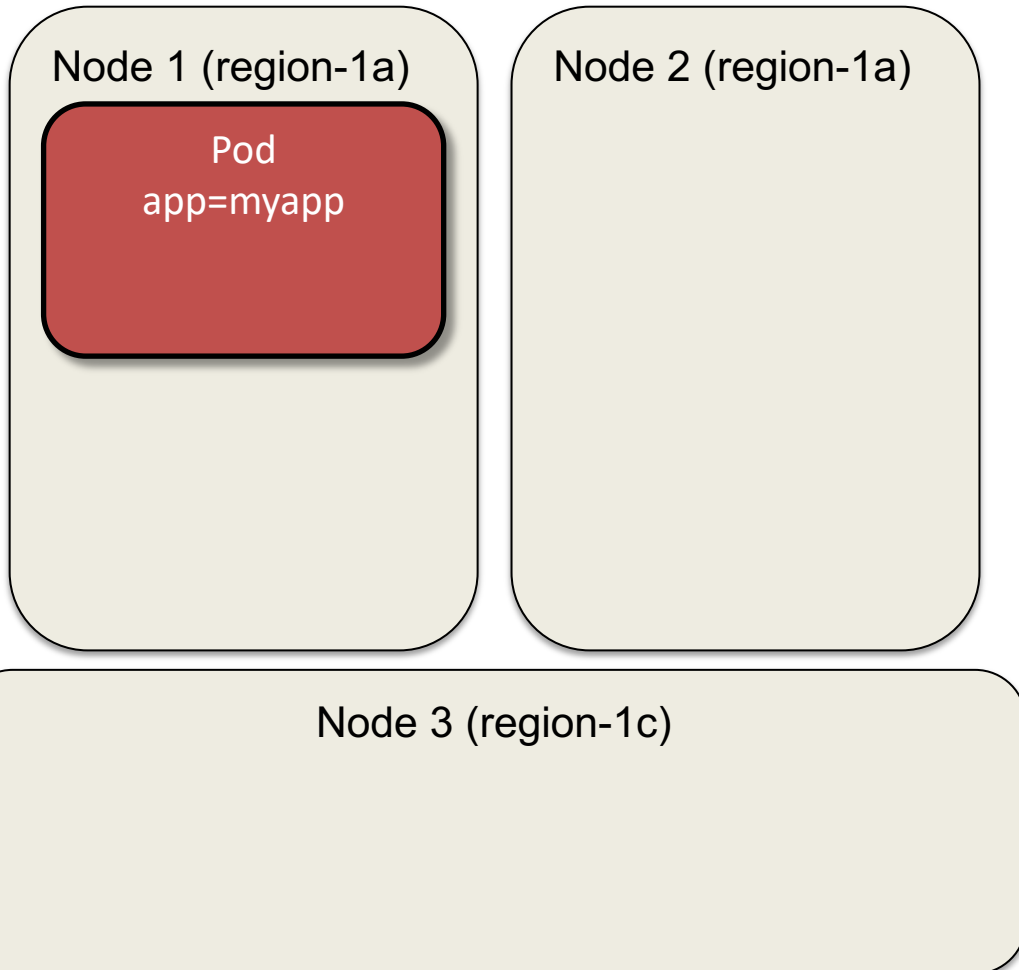
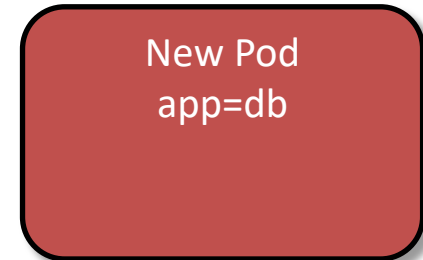
- Schedule Pods onto nodes that have same labels
 - app=myapp
 - operator: In
 - kubernetes.io/hostname



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

Pod Affinity

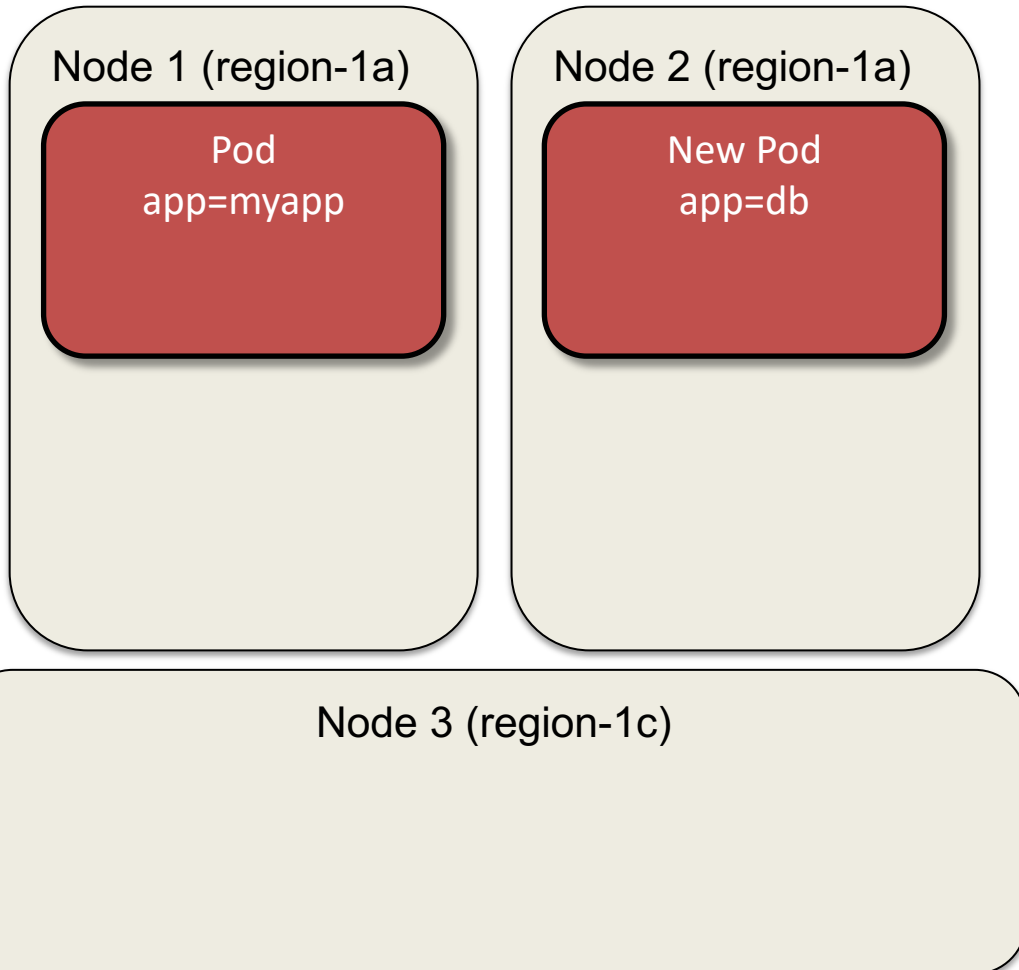
- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: " failure-domain.beta.kubernetes.io/zone"
```

Pod Affinity

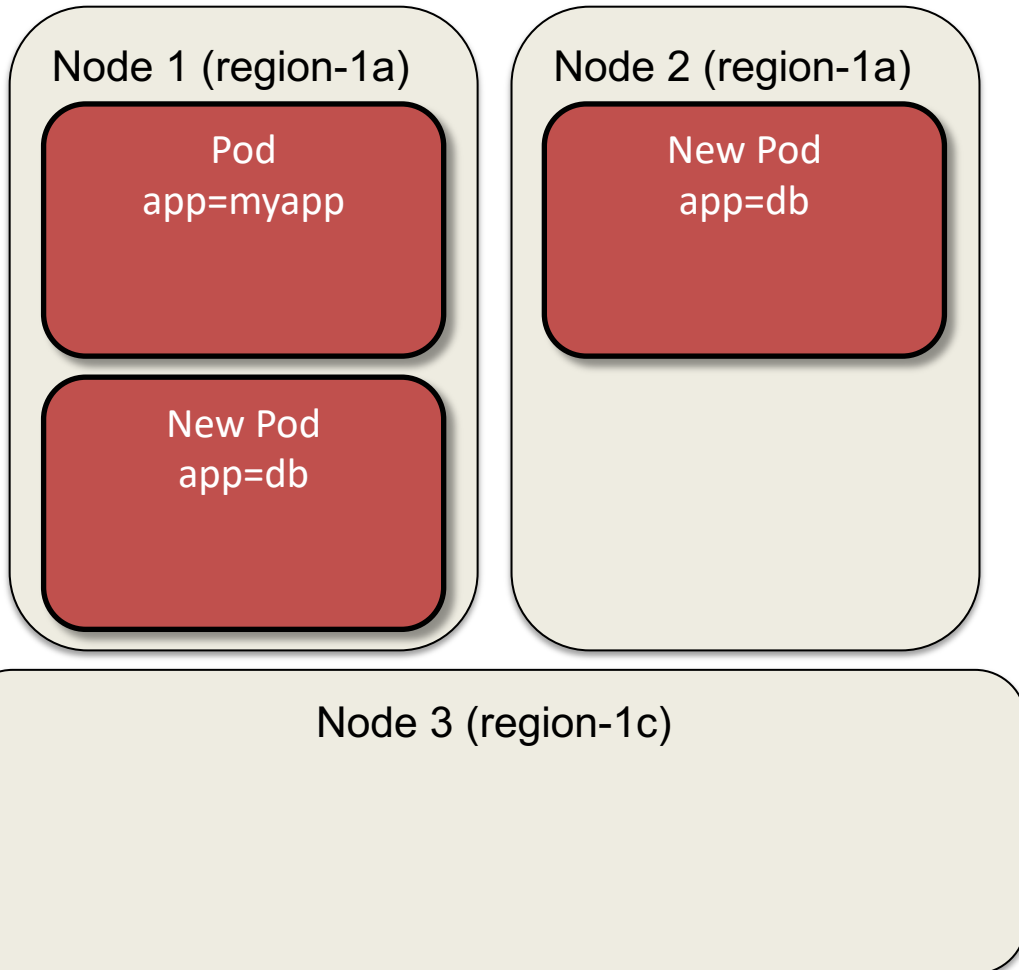
- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: " failure-domain.beta.kubernetes.io/zone"
```


Pod Affinity

- Schedule Pods onto nodes in same region
 - app=db
 - failure-domain.beta.kubernetes.io/zone



```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: " failure-domain.beta.kubernetes.io/zone"
```

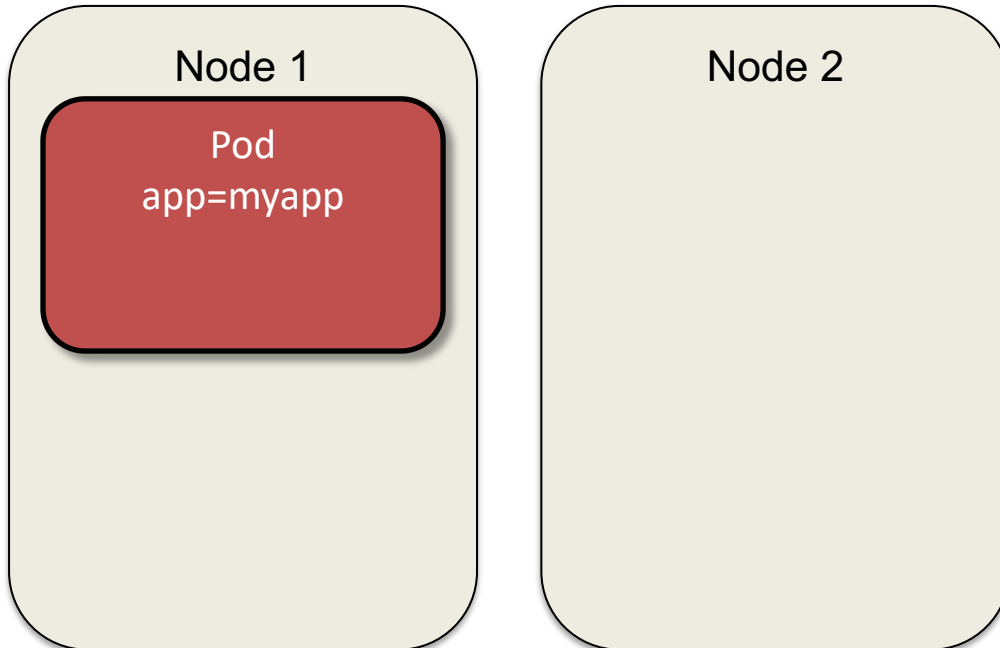
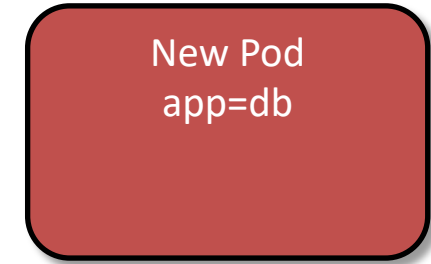
Pod Anti-Affinity

- Define how pods should be placed relative to one another
- Pods of different services run on different nodes.

```
affinity:  
  podAntiAffinity:  
    preferredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: service  
              operator: In  
              values: ["S1"]  
        topologyKey: kubernetes.io/hostname
```

Pod Anti-Affinity

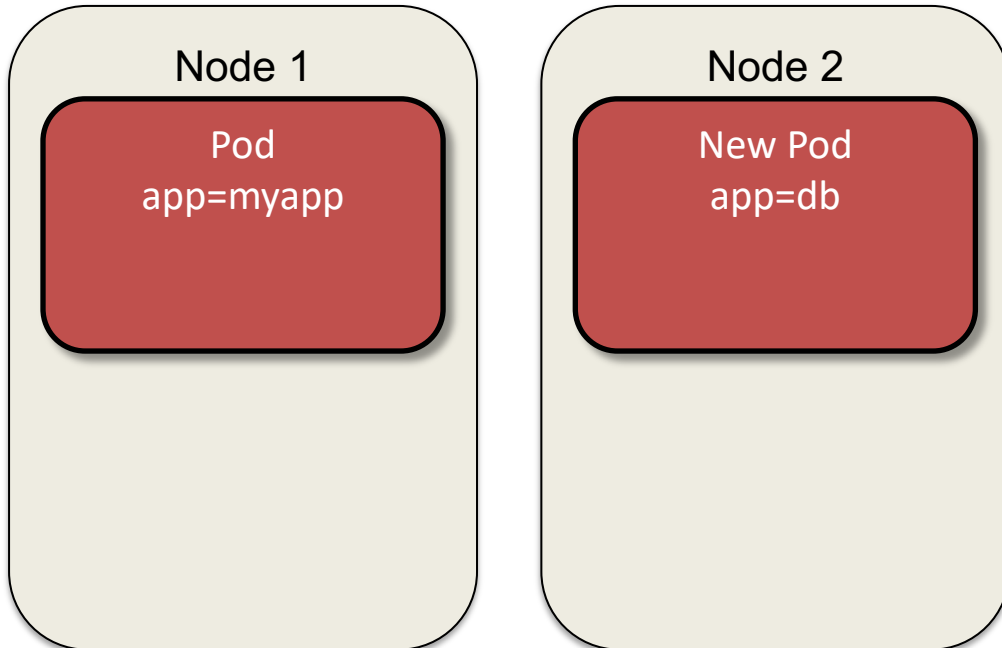
- Do not schedule Pods onto nodes with matching labels.
 - app=db
 - operator: In
 - kubernetes.io/hostname



```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
            - key: "app"
              operator: In
              values:
                - myapp
        topologyKey: "kubernetes.io/hostname"
```

Pod Anti-Affinity

- Do not schedule Pods onto nodes with matching labels.
 - app=db
 - operator: In
 - kubernetes.io/hostname



```
affinity:  
  podAntiAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      - labelSelector:  
          matchExpressions:  
            - key: "app"  
              operator: In  
              values:  
                - myapp  
        topologyKey: "kubernetes.io/hostname"
```

Lab: Scheduling



Deployment Strategies Overview



What is a Deployment Strategy?

Approaches to manage risks on updating Deployments

- On each Deployment update/change, all pods in the deployment will be deleted and recreated
- Recreation process can have service impacts, especially for large Deployments
- A Deployment strategy defines how this rebuild process is done, to minimize downtime due to application failures or malfunctions

Types of Deployment Strategies

Kubernetes supports two basic strategies, but users can also leverage multiple Deployments when applying changes

- Strategies for single Deployments
 - Recreate
 - RollingUpdate
- Strategic approaches using two Deployments with a Service
 - Canary deployments
 - Blue/Green deployments

Each approach has a specific behavior and advantages/disadvantages.

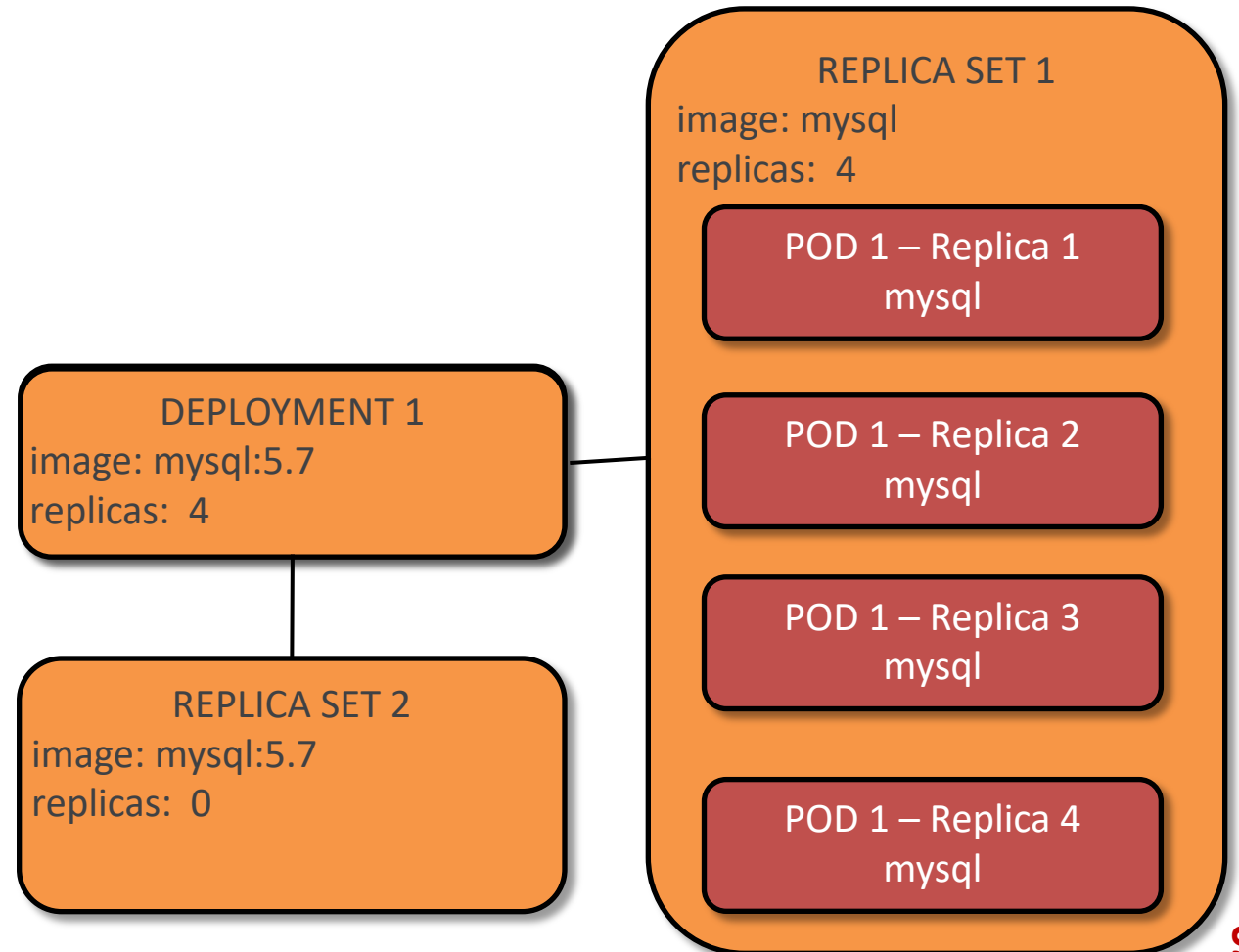
Deployment Strategy: Recreate



Deployment Strategy: Recreate

Simplest strategy for deployments

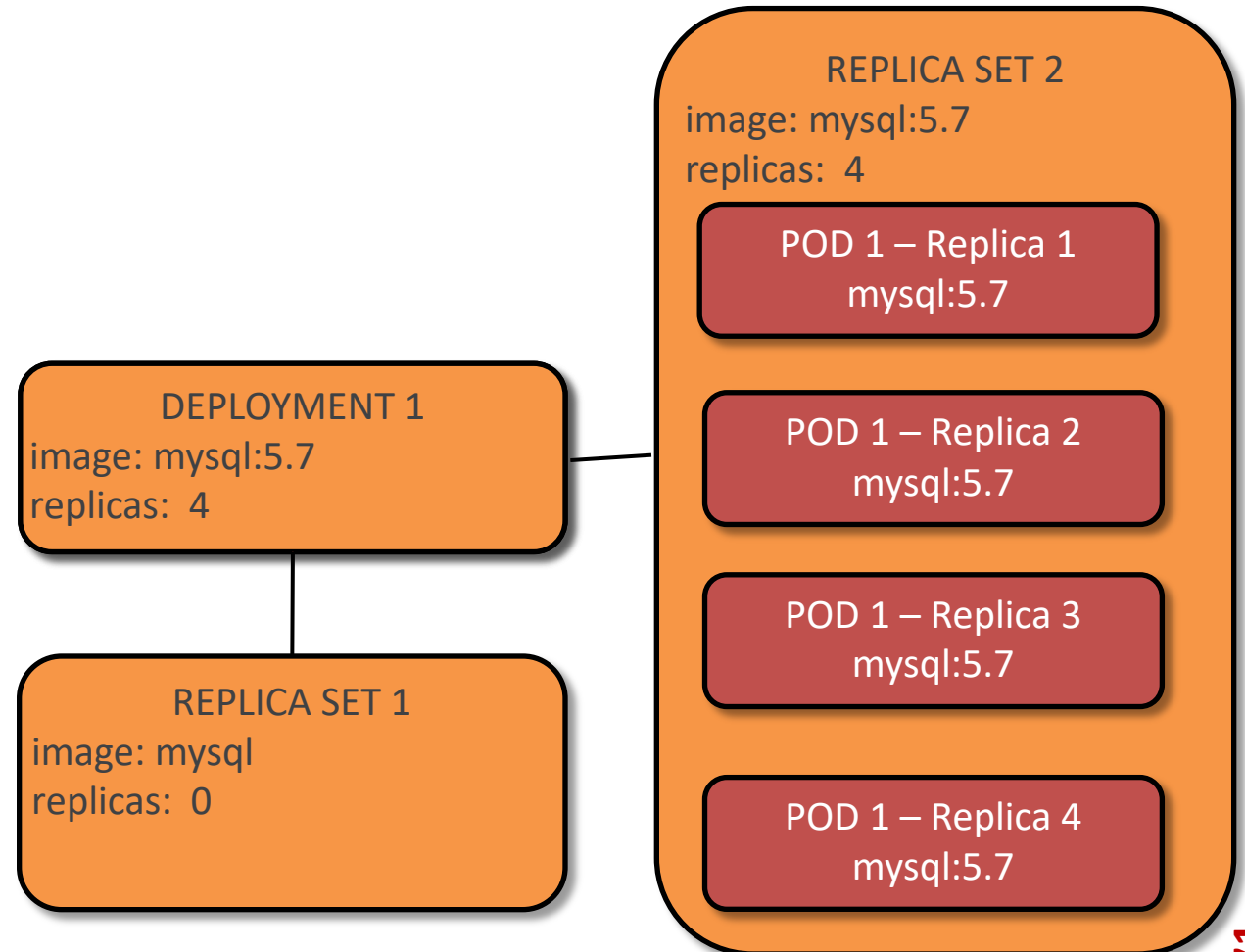
- When a change is made to a Deployment's spec, all Pods are removed and then recreated
 - Old Replica Set pods are killed
 - Then, new Replica Set starts pods
- May lead to downtime during the process while new pods are started



Deployment Strategy: Recreate

Simplest strategy for deployment updates

- When a change is made to a Deployment's template, all Pods are removed and then recreated
 - Old Replica Set pods are killed
 - New Replica Set starts pods
- May cause downtime due to delay between old pods terminating and new pods becoming available



Deployment Strategy: Recreate

Strategies are defined in the spec of a Deployment

- **strategy** parameter in Deployment spec sets the strategy to be used for updates
- If no parameter value is set, the default is **RollingUpdate**

```
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: tomcat-deployment
spec:
  replicas: 3
  strategy:
    type: Recreate
  template:
    metadata:
      labels:
        type: webserver
    spec:
      containers:
      - name: tomcat-container
        image: tomcat
        ports:
        - containerPort: 8080
```

Deployment Strategy: RollingUpdate



Deployment Strategy: RollingUpdate

RollingUpdate is DEFAULT strategy for Deployments

- When a change is made to the Deployment, the old Replica Set pods are scaled down as new pods are created by the new Replica Set
- A minimum number of running Pods is specified, so the Deployment will never be totally out of Pods to respond to service requests
- During the update process, the requested replica count may be temporarily exceeded

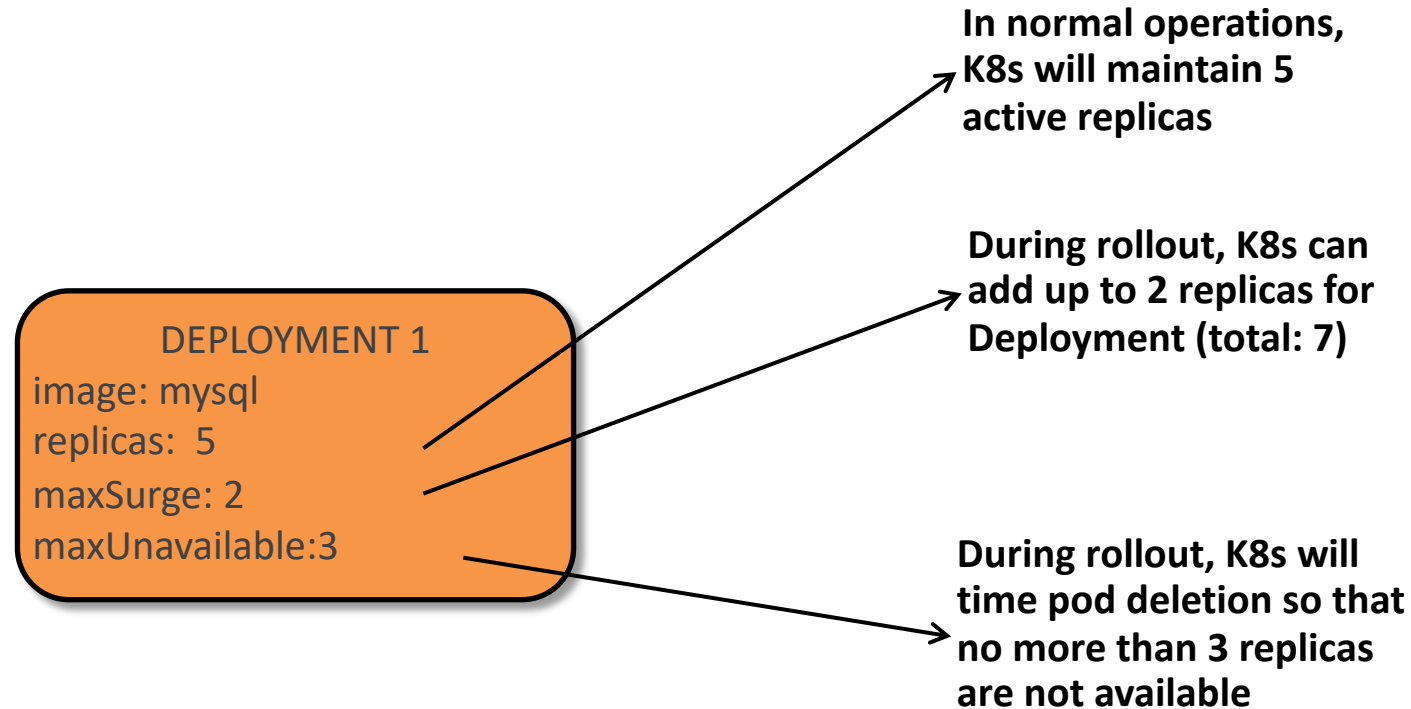
Deployment Strategy: RollingUpdate

Configure parameters to control the update process

```
...
metadata:
  name: tomcat-deployment
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 10%
  template:
    metadata:
      labels:
        type: webserver
...
```

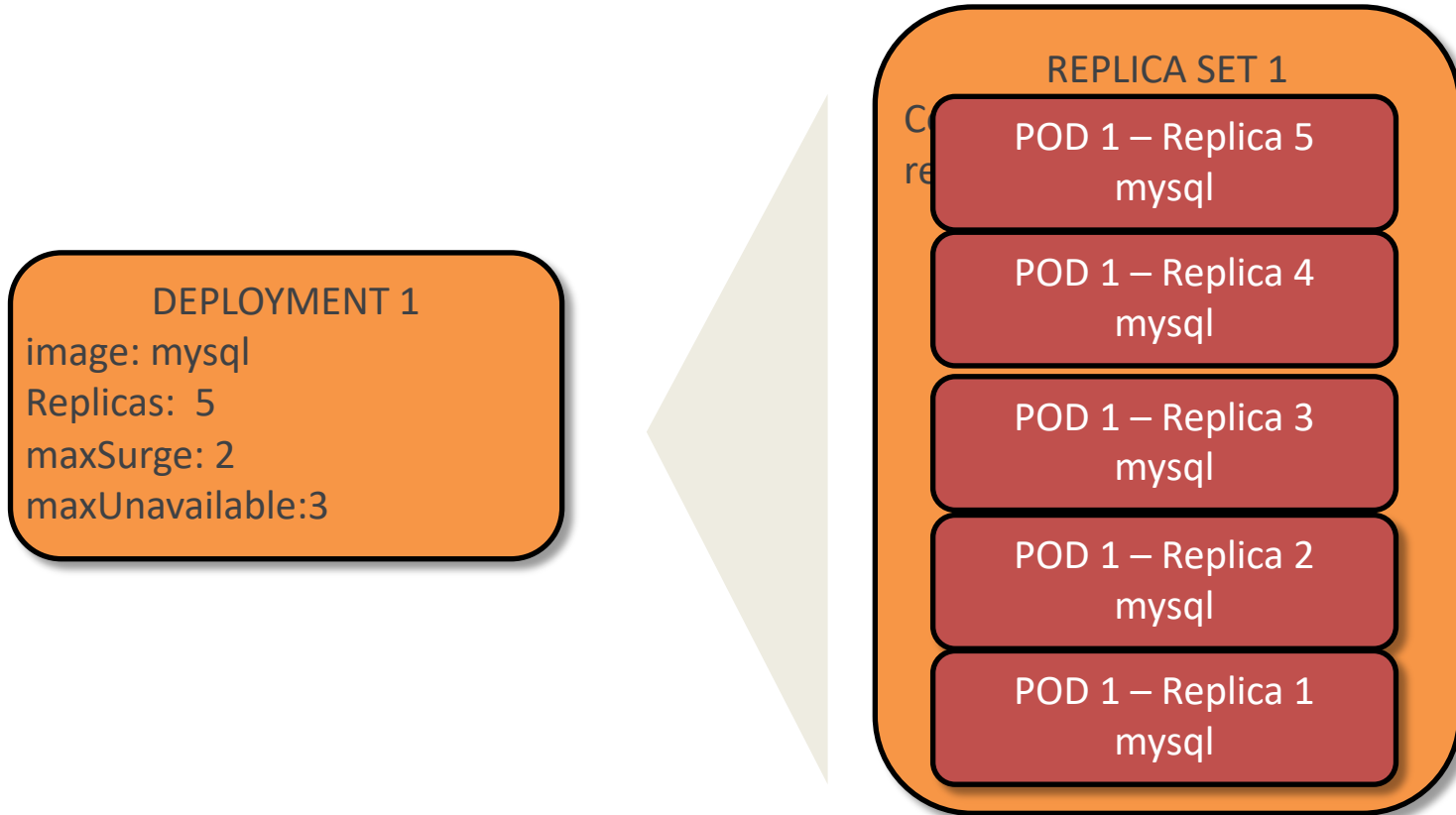
- **maxSurge**: number or percentage of additional Pods that can be created exceeding the replica count during update
 - Default value of 25%
- **maxUnavailable**: number of Pods that can be unavailable during the update
 - Default value of 25%

Deployment Strategy: RollingUpdate



Deployment Strategy: RollingUpdate

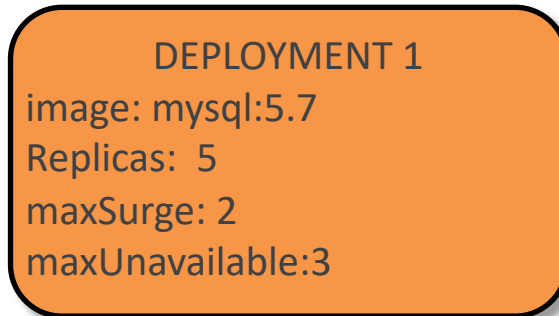
Initial State



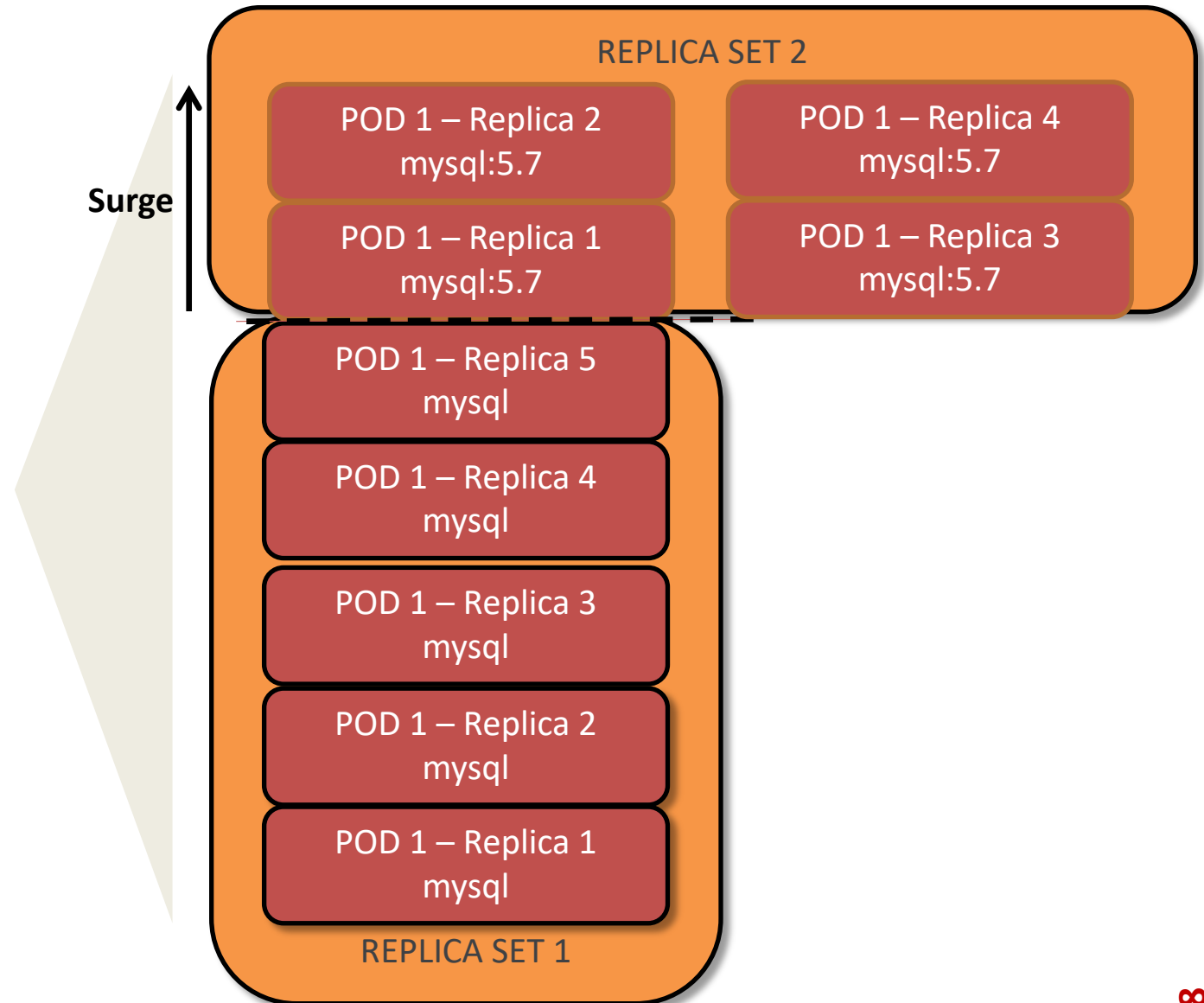
```
$ vi simpledeployment.yaml
...
  image: mysql:5.7
...
$ kubectl apply -f simpledeployment.yaml
```

Deployment Strategy: RollingUpdate

Rollout in progress

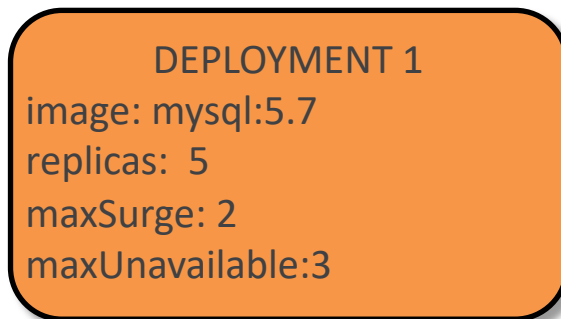


- Initial surge of new pods on new Replica Set
- Original Replica Set scaled back as new RS scaled out

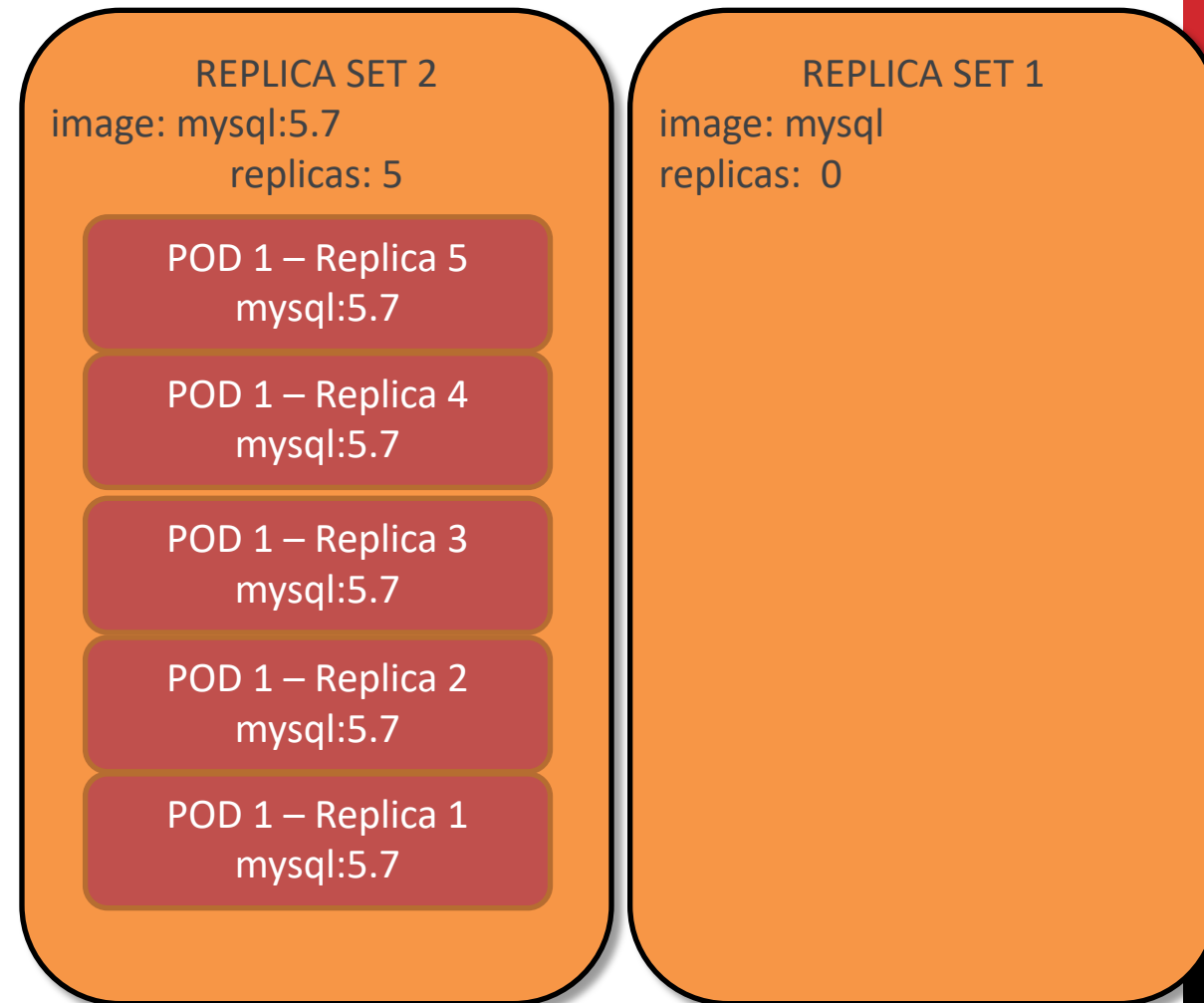


Deployment Strategy: RollingUpdate

Rollout
complete



- By default, old, inactive Replica Set saved – previous version of the Deployment



Updating Using Multiple Deployments



RollingUpdate using Multiple Deployments

Controlled testing of new versions in production

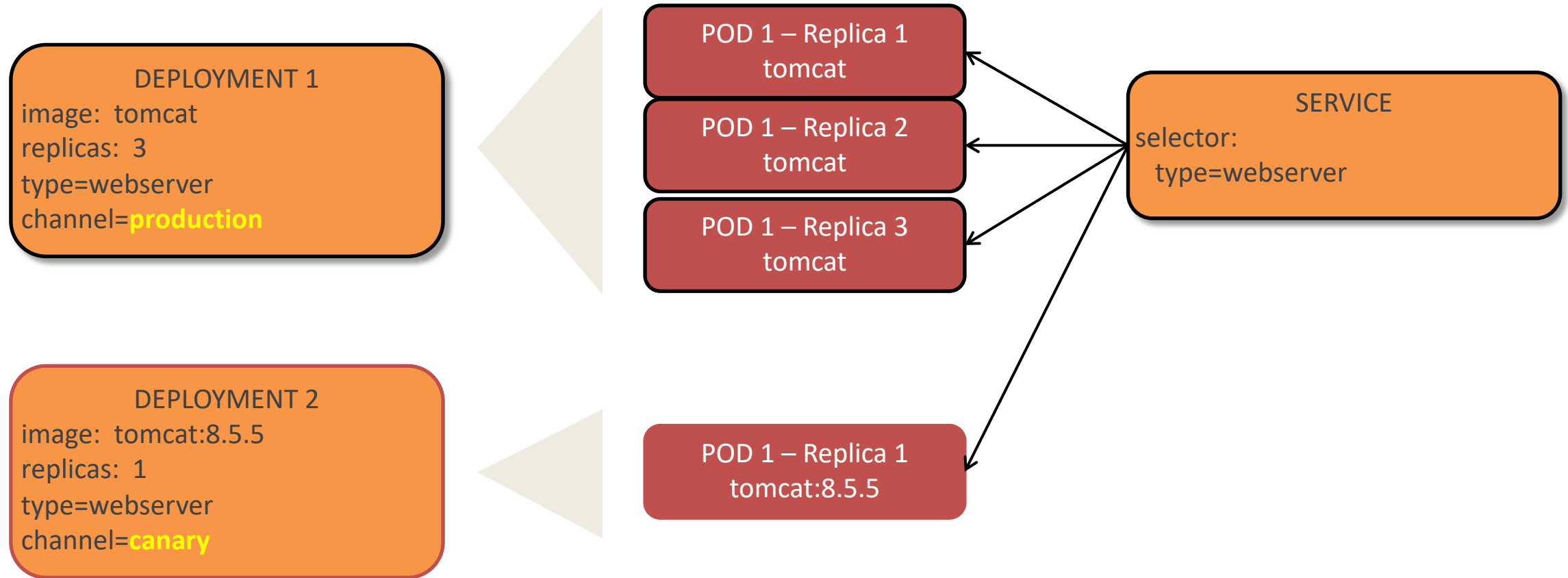
- Assume an application running as a Deployment, exposed as a Service
- To apply a new application version in production, a second Deployment can be used using labels in common with the first Deployment
 - Canary deployment allows for limited testing of new version in production
 - Blue/green deployment

Strategic Approach: Canary Deployment

Controlled testing of the update on production

- Consider a Service selecting pods from a Deployment of application pods
- In a canary deployment, a second Deployment (Canary Deployment) is created with pods for the new version, with labels matching the Service's selector
- Service directs some requests to pods on the Canary, allowing testing of changes in production
- If a malfunction is detected, it will only impact a small portion of the Pods and can be undone.

Strategic Approach: Canary Deployment



Strategic Approach: Canary Deployment

Decisions after running the canary Deployment in production

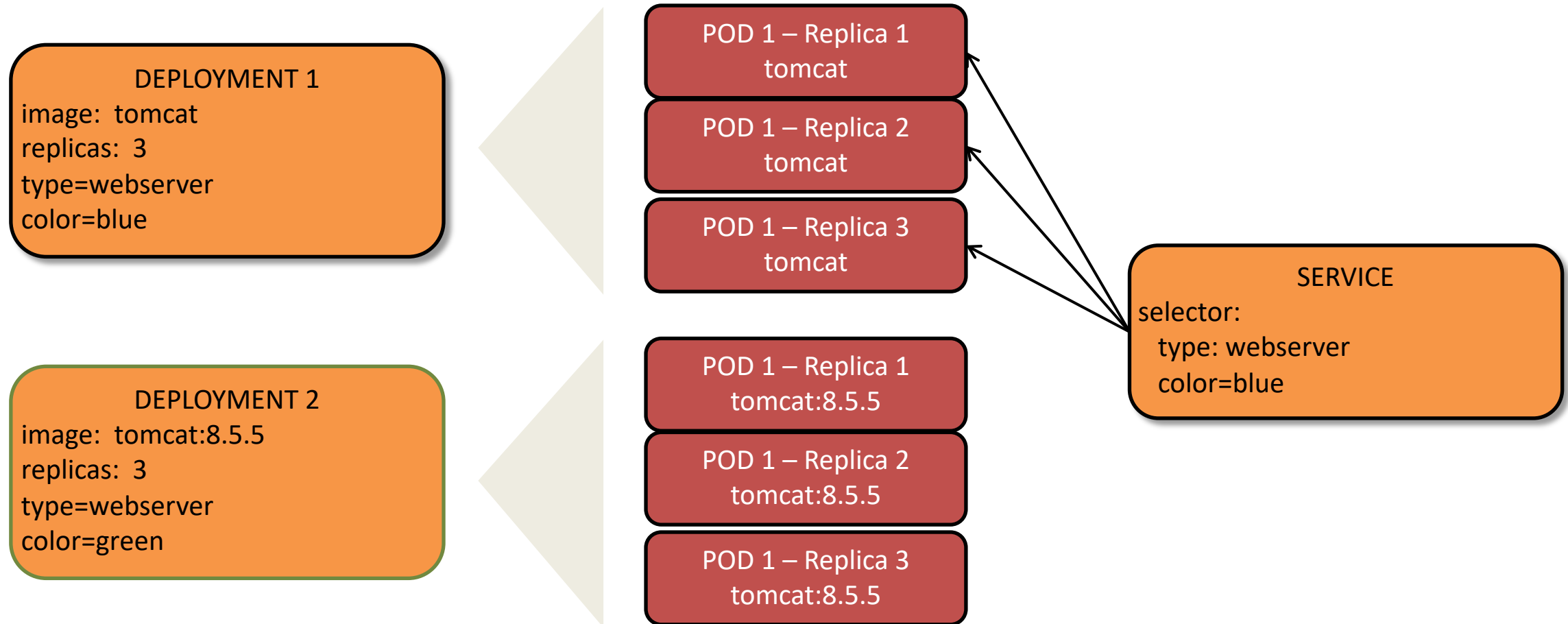
- If the application error rate is not increasing and Canary Deployment is stable:
 - The main Deployment can be updated to the newer version (using Rolling Update for example) and then the Canary can be discarded; OR
 - The Canary can be scaled up and reconfigured and the old Deployment can be discarded.
- If the test results in failure, the Canary deployment can be deleted

Strategic Approach: Blue/Green Deployment

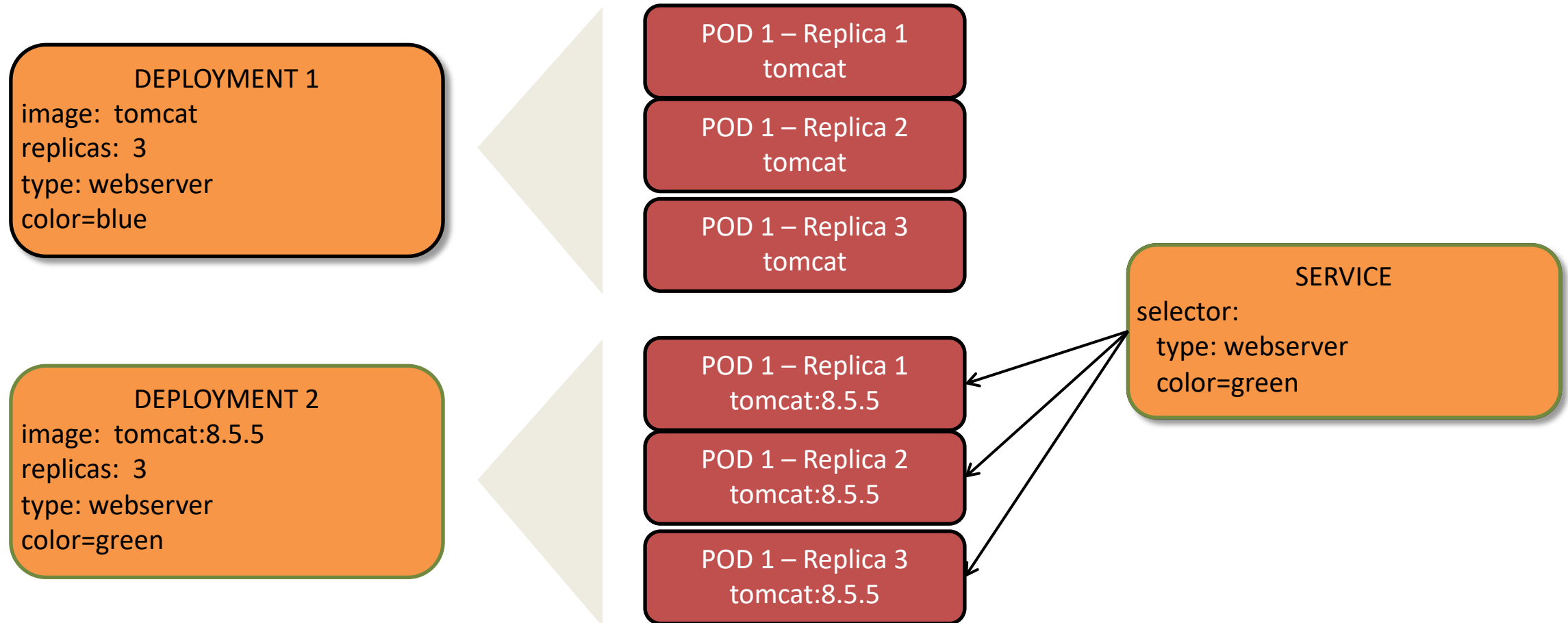
Complete environment switch from one version to another

- With a Blue/Green deployment, you create a new full-scale Deployment in addition to the current production Deployment
- Reconfiguring the pod label selector on the application's Service allows choice of directing requests to old Deployment or new Deployment
- Similar to effect of Replace strategy without application downtime

Strategic Approach: Blue/Green Deployment



Strategic Approach: Blue/Green Deployment



Questions



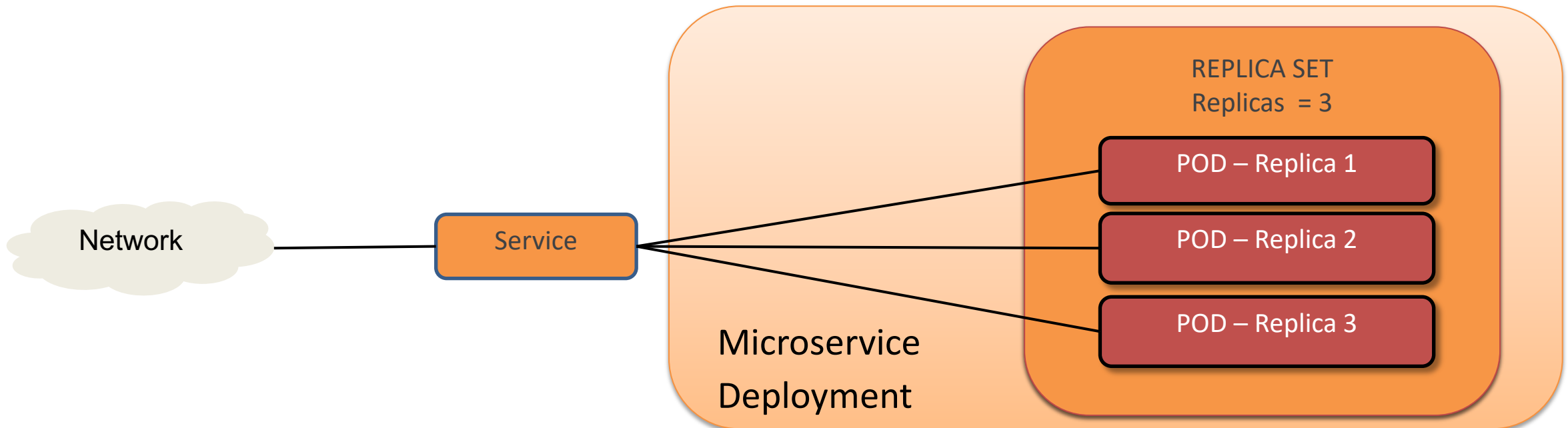
Horizontal Pod Autoscaler (HPA)



Kubernetes Deployments for Microservices

Kubernetes is perfect for Microservices management

- Applications built as services or collections of microservices easily modeled using Kubernetes Deployment and Service resources
- Key property is that application or component can be scaled horizontally



Basic Microservices Principles

- Application is composed of in(ter)dependent, independently deployable components
 - E.g. containerized components packaged as Docker images
- Design of application is service-oriented, with information exchange between components via standard interfaces, e.g.
 - RESTful APIs
 - AMQP messaging
- Maximize use of stateless components that can be scaled horizontally via replication
 - Horizontal scaling provides elastic capacity for handling demand

Application Scaling Strategies

Several ways to achieve scalability

- **Pre-defined application scale:** application deployed with a predefined, static number of resources => not the Kubernetes way
- **Manual scaling:** deployment scale reconfiguration driven by operator
- **Auto-scaling:** automatic scaling of resources based on a defined trigger (number of hits, CPU usage, etc)

Application Scaling Strategies

Manual scaling

- K8s supports manual scaling of Deployment to adjust replica count
- Operator just needs to adjust declaration of how many replicas are desired, and K8s system will manage to that number
 - Pod creation and placement completely automatic
 - Pods automatically re-created if nodes fail

```
$ kubectl scale --replicas=2 deployment/tomcat  
$ kubectl scale --replicas=8 deployment/tomcat  
$ kubectl scale --replicas=4 deployment/tomcat
```

```
$ kubectl edit deployment/tomcat
```

```
$ kubectl apply -f simpledeployment.yaml
```

Application Scaling Strategies

Auto-scaling in Kubernetes

- Kubernetes has a controller object for automatic scaling of Deployments (or Replica Sets or ReplicationControllers)
- HorizontalPodAutoscaler is control loop to adjust scale of pod set depending on one or more metrics, evaluated at configurable interval (default 30s)
 - Requires metrics-server to be deployed on the cluster to provide metric data
 - Metrics include CPU and RAM utilization for pods
- Typical scenario: increase Deployment replica count when average Pod CPU utilization is above threshold value for specified period of time
- Note: application must support horizontal scaling!

Application Scaling Strategies

Auto-scaling in Kubernetes

- HPA will either reduce or increase the number of pods in the set to a level that will enable each pod in the set to match the desired baseline usage as closely as possible. The baseline is calculated based on the CPU limit in the pod specifications, for example:

```
resources:  
  requests:  
    cpu: 25m  
  limits:  
    cpu: 100m
```

Automatic Scaling of a Deployment

Creating a HorizontalPodAutoscaler resource

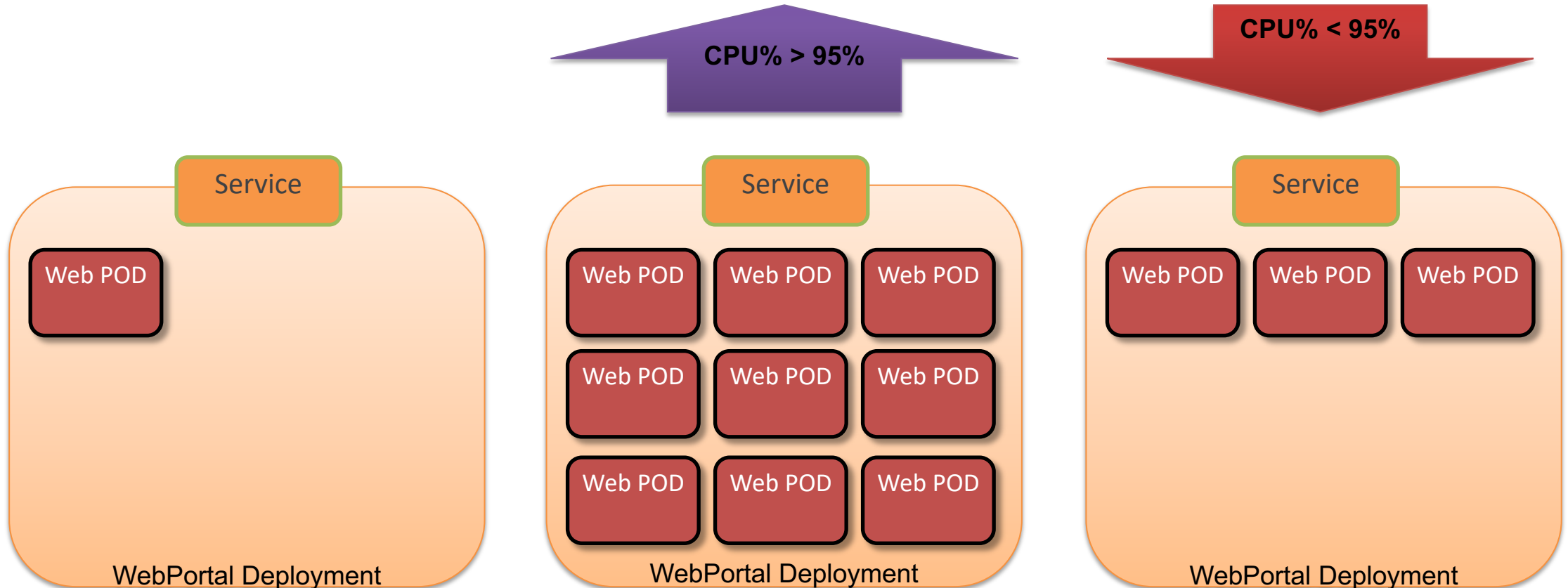
- The hpa is created similarly to any other Kubernetes resource
 - Use a manifest file for source tracking
 - Can create directly with *kubectl*
- 30 seconds is the default for considering the threshold
- Scaling is metrics-driven, either resource metrics from metrics-server, custom metrics API, or external metrics API

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: tomcat-autoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: tomcat-deployment
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 50
```

simplescaler.yaml

Auto-Scaling of a Deployment

HorizontalPodAutoscaler with a 95% CPU usage threshold



Lab: Horizontal Pod Autoscaler

